



RNN,
LSTM,
GRU,
Transformer

법률 고지사항

- Intel® 디지털 준비 프로그램 및 Intel® AI for Future Workforce 프로그램은 Intel Corporation에서 개발했습니다.
- © Intel Corporation. Intel, Intel 로고 및 기타 Intel 마크는 Intel Corporation 또는 자회사의 상표입니다. 다른 이름 및 브랜드는 다른 사람의 재산으로 주장될 수 있습니다. 프로그램 날짜와 수업 계획은 변경될 수 있습니다.
- Intel 기술에는 활성화된 하드웨어, 소프트웨어 또는 서비스 활성화가 필요할 수 있습니다.
- 모든 제품과 구성 요소는 안전을 보장 할 수 없습니다.
- 결과물은 추정되거나 시뮬레이션 되었습니다.
- Intel은 타사 데이터를 제어하거나 감사하지 않습니다. 정확성을 평가하려면 다른 출처를 참조해야 합니다.
- 당신이 투자한 비용과 그에 대한 결과물은 다를 수 있습니다.

RNN, LSTM and GRU

AI for Future Workforce

일반 데이터

차	주행거리	마력	용량(CC)	가격
Car1	49,666	90	2,000	13,500
Car2	19,700	192	1,800	21,500
Car3	31,461	192	1800	20,950
Car4	43,600	192	1,800	19,950

시계열 데이터

- 시간의 흐름에 따라 관측되어 시간의 영향을 받게 되는 데이터
- 현재 상태가 과거의 영향을 받음

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량
15:00	68	98.9	7.9	정상
16:00	-0.9	72.3	6.6	정상
17:00	-1	79.2	6.6	불량

다변량 시계열 데이터

시계열 데이터 분석 방법론 트렌드

- 머신러닝 기반 시계열 데이터 분석 방법론
 - Linear regression / Logistic regression model
 - Support vector machine / regression
 - Random forest
 - Boosting (ex, Gradient Boosting)
- Neural Network 기반 시계열 데이터 분석 방법론
 - Recurrent neural networks (RNN)
 - LSTM
 - GRU
 - Seq2Seq with attention
 - Convolutional neural networks (CNN for time series analysis)
 - Transformer
 - GPT-1, BERT, GPT-3, GPT-3.5, ChatGPT

Neural Network 모델

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상

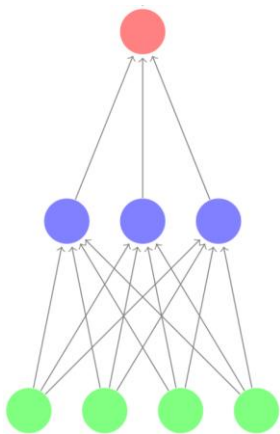
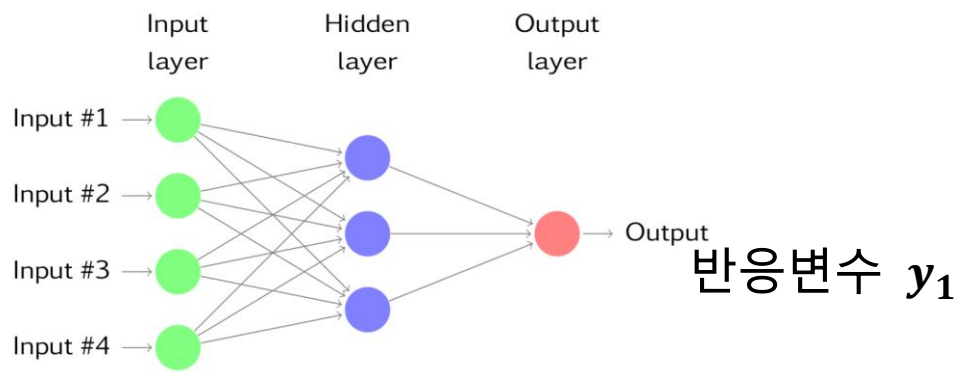
T(시간)=1

$$h_1 = f(W_{xh} x_1)$$

$$y_1 = g(W_{hy} h_1)$$

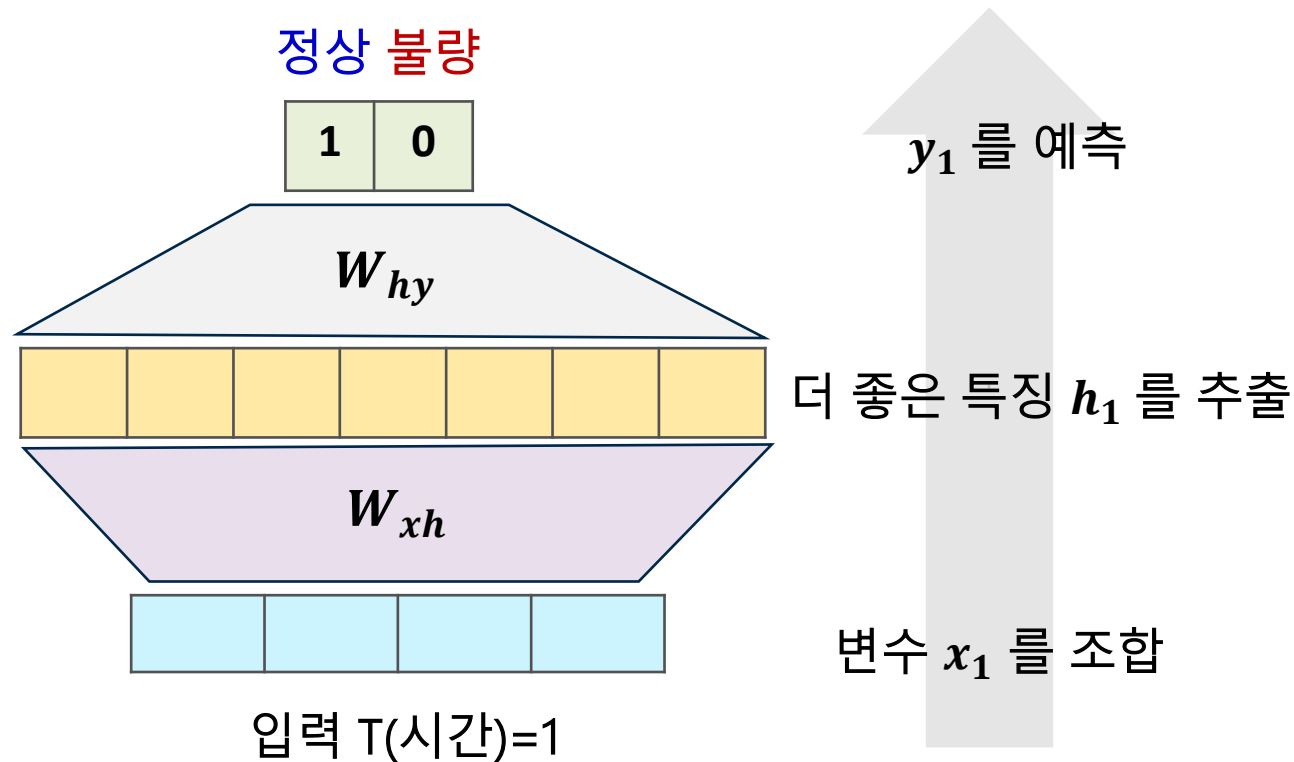
$$f(\cdot) = \tanh$$

$$g(\cdot) = \text{softmax}$$



은닉층 h_1

입력 변수 x_1



Neural Network 모델

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상

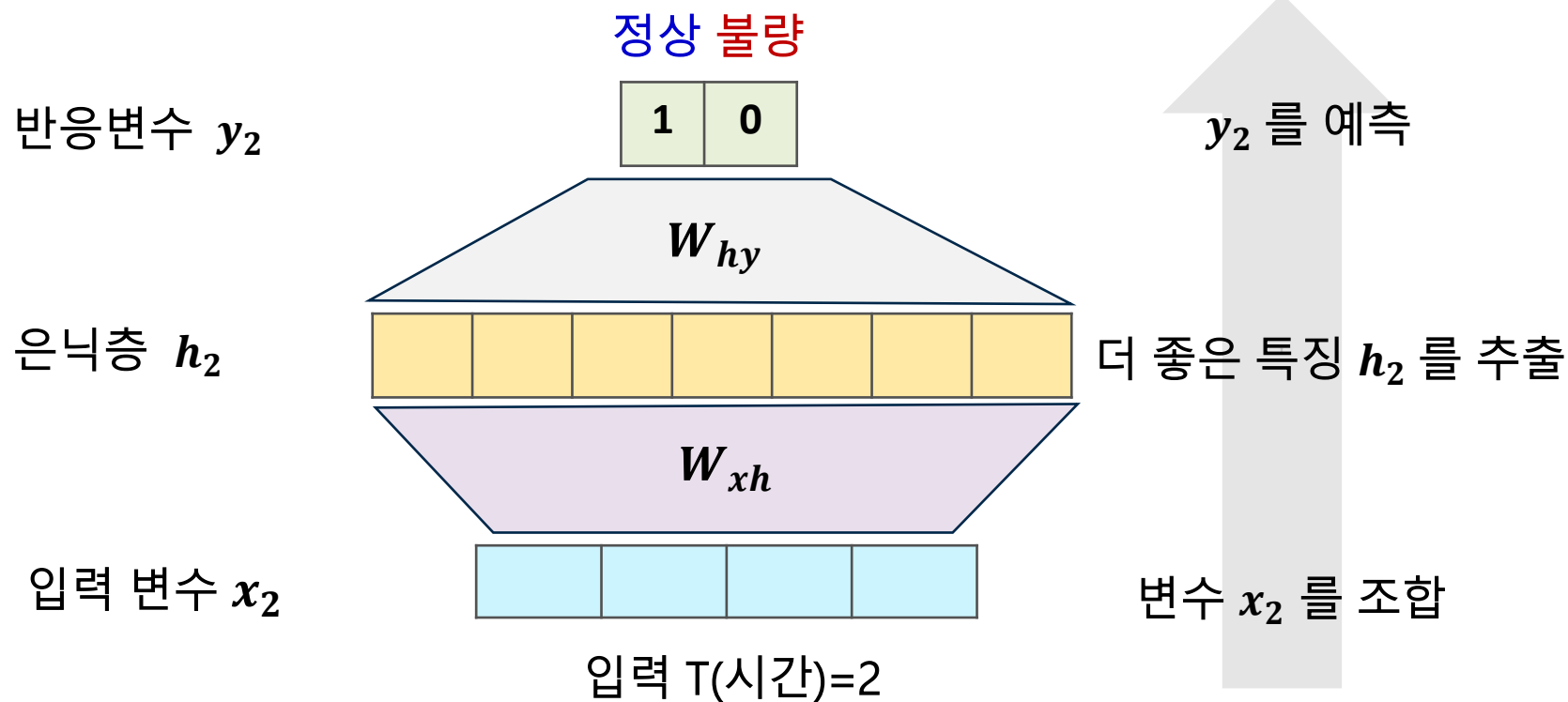
T(시간)=2

$$h_2 = f(W_{xh} x_2)$$

$$y_2 = g(W_{hy} h_2)$$

$$f(\cdot) = \tanh$$

$$g(\cdot) = \text{softmax}$$



Neural Network 모델

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

$t-2$

$t-1$

t

$$h_t = f(W_{xh} x_t)$$

$$y_t = g(W_{hy} h_t)$$

$$f(\cdot) = \tanh$$

$$g(\cdot) = \text{softmax}$$

이전 시점 정보를 반영하지 않음

반응변수 y_t

정상 불량

1 0

은닉층 h_t

W_{hy}

더 좋은 특징 h_t 를 추출

입력 변수 x_t

W_{xh}

변수 x_t 를 조합

입력 T(시간) = t

y_t 를 예측

순환신경망(Recurrent Neural Network) 필요성

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

$$h_t = f(W_{xh} x_t) + \text{이전 시점 정보}$$

$$y_t = g(W_{hy} h_t)$$

$t-2$

$t-1$

t

$$f(\cdot) = \tanh$$

$$g(\cdot) = \text{softmax}$$

이전 시점 정보를 반영해서 더 나은 예측을 하자!

반응변수 y_t

정상 불량

1 0

은닉층 h_t

W_{hy}

더 좋은 특징 h_t 를 추출

입력 변수 x_t

W_{xh}

변수 x_t 를 조합

입력 T(시간) = t

y_t 를 예측

순환신경망(Recurrent Neural Network) 필요성

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

$$h_t = f(W_{xh} x_t) + \text{이전 시점 정보}$$

$$y_t = g(W_{hy} h_t)$$

$t-2$

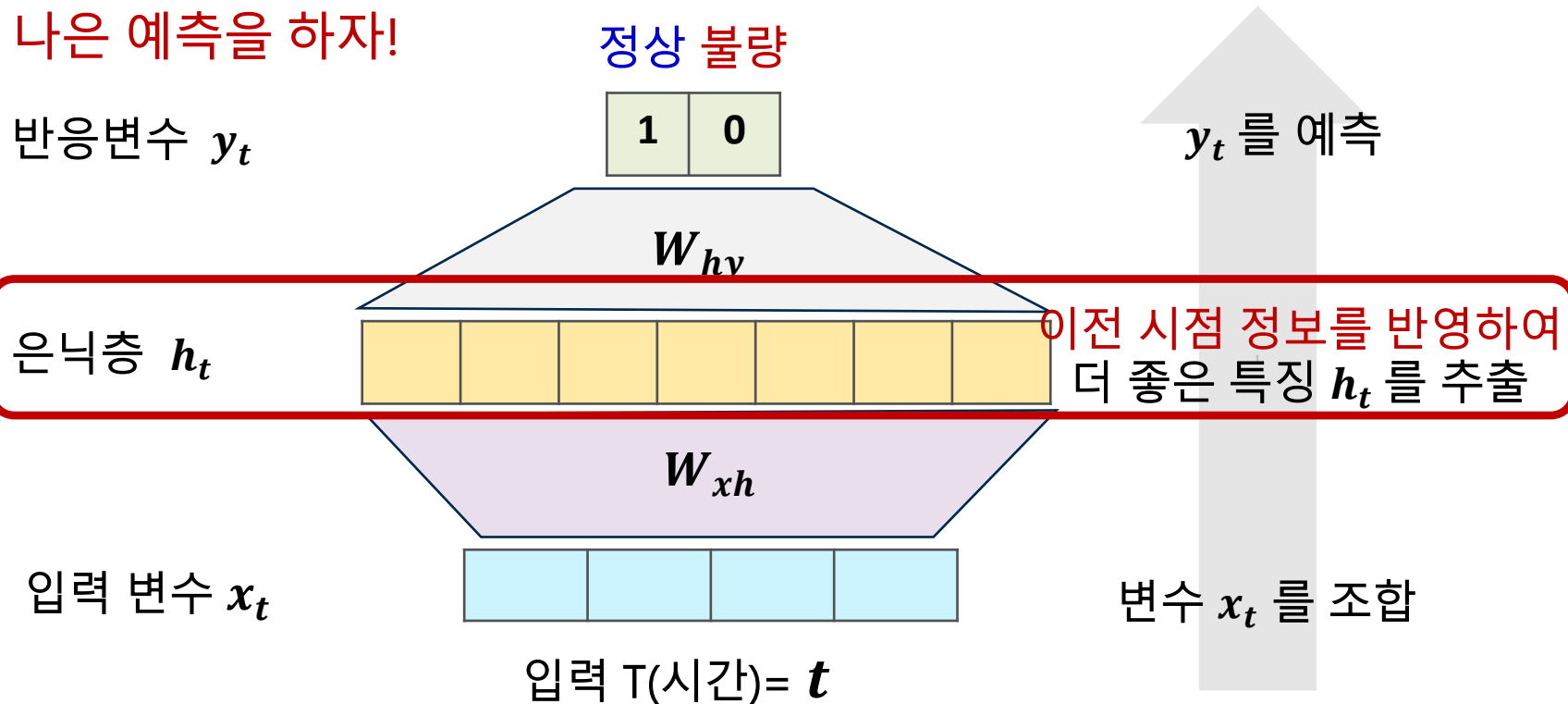
$t-1$

t

$$f(\cdot) = \tanh$$

$$g(\cdot) = \text{softmax}$$

이전 시점 정보를 반영해서 더 나은 예측을 하자!



순환신경망(Recurrent Neural Network) 필요성

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

$$h_t = f(W_{xh} x_t) + t-1 \text{ 정보}$$

$$y_t = g(W_{hy} h_t)$$

$t-1$

t

$$f(\cdot) = \tanh$$

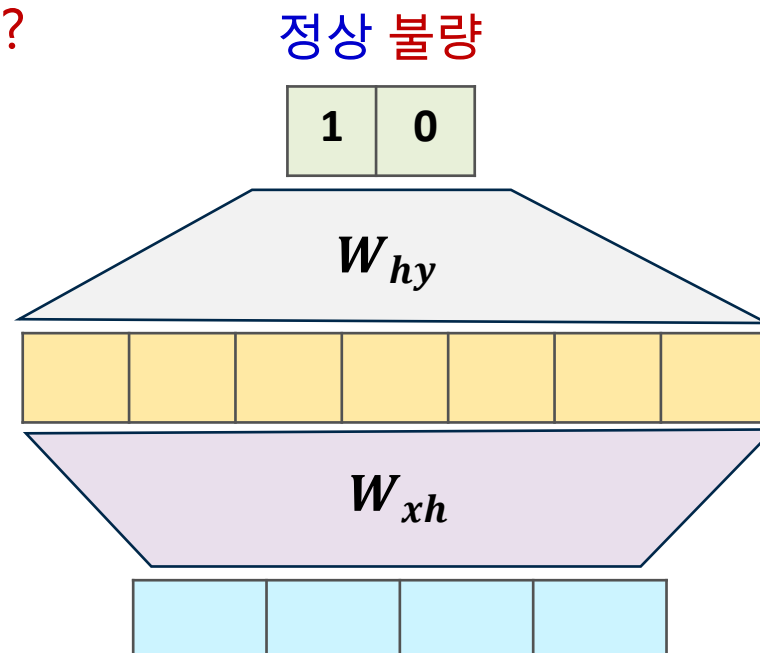
$$g(\cdot) = \text{softmax}$$

h_t 에 이전 시점 $t-1$ 정보를 반영하는 방법은?

반응변수 y_t

은닉층 h_t

입력 변수 x_t



입력 T(시간)= t

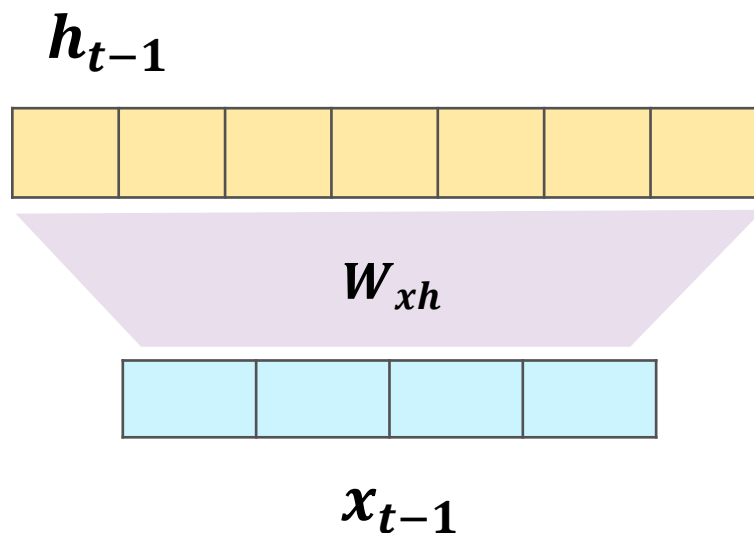
순환신경망(Recurrent Neural Network) 필요성

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

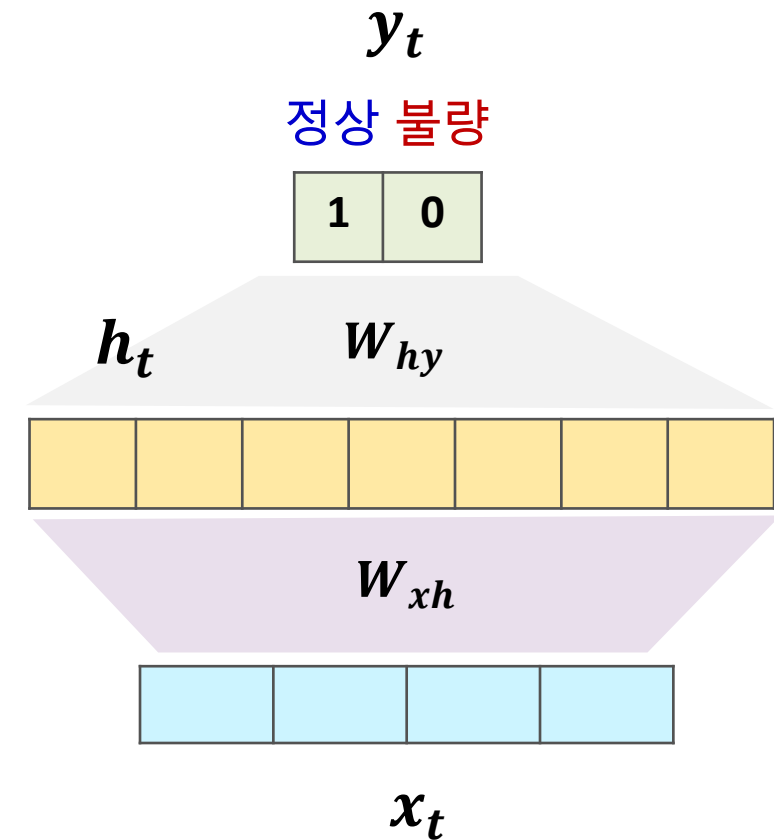
$$h_t = f(W_{xh} x_t) + \text{t-1 정보}$$

$$y_t = g(W_{hy} h_t)$$

$$f(\cdot) = \tanh \quad g(\cdot) = \text{softmax}$$



h_t 에 h_{t-1} 를 합성



순환신경망(Recurrent Neural Network) 필요성

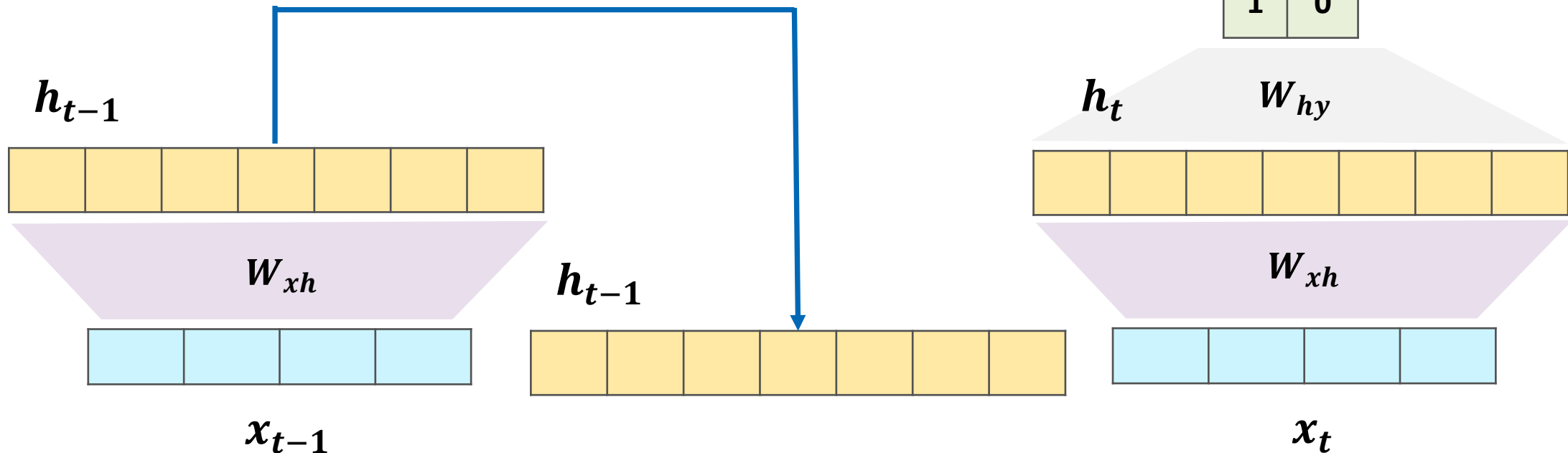
시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

$$h_t = f(W_{xh} x_t + W_{hh} h_{t-1})$$

$$y_t = g(W_{hy} h_t)$$

$t-2$
 $t-1$
 t

$$f(\cdot) = \tanh \quad g(\cdot) = \text{softmax}$$



순환신경망(Recurrent Neural Network) 필요성

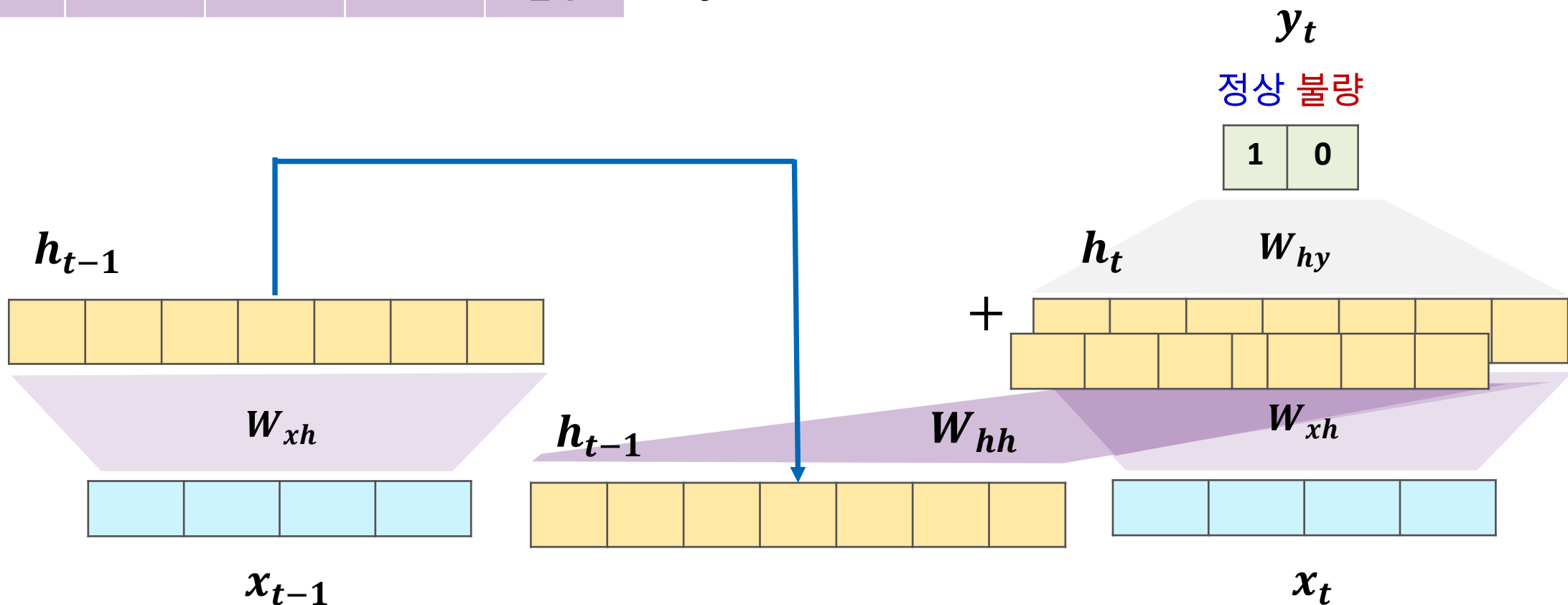
시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

$$h_t = f(W_{xh} x_t + W_{hh} h_{t-1})$$

$$y_t = g(W_{hy} h_t)$$

$t-2$
 $t-1$
 t

$$f(\cdot) = \tanh \quad g(\cdot) = \text{softmax}$$



순환신경망(Recurrent Neural Network) 필요성

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

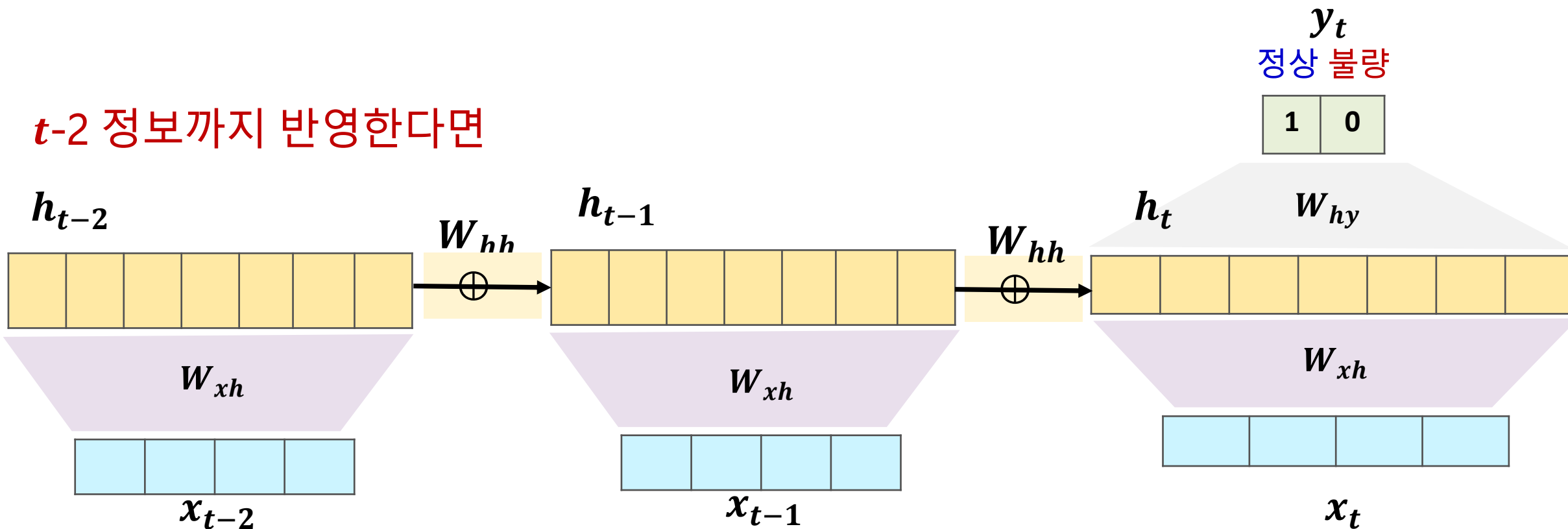
$$h_{t-1} = f(W_{xh} x_{t-1} + W_{hh} h_{t-2})$$

$$t-2 \quad h_t = f(W_{xh} x_t + W_{hh} h_{t-1})$$

$$t-1 \quad y_t = g(W_{hy} h_t)$$

$$t \quad f(\cdot) = \tanh \quad g(\cdot) = \text{softmax}$$

$t-2$ 정보까지 반영한다면



순환신경망(Recurrent Neural Network) 필요성

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

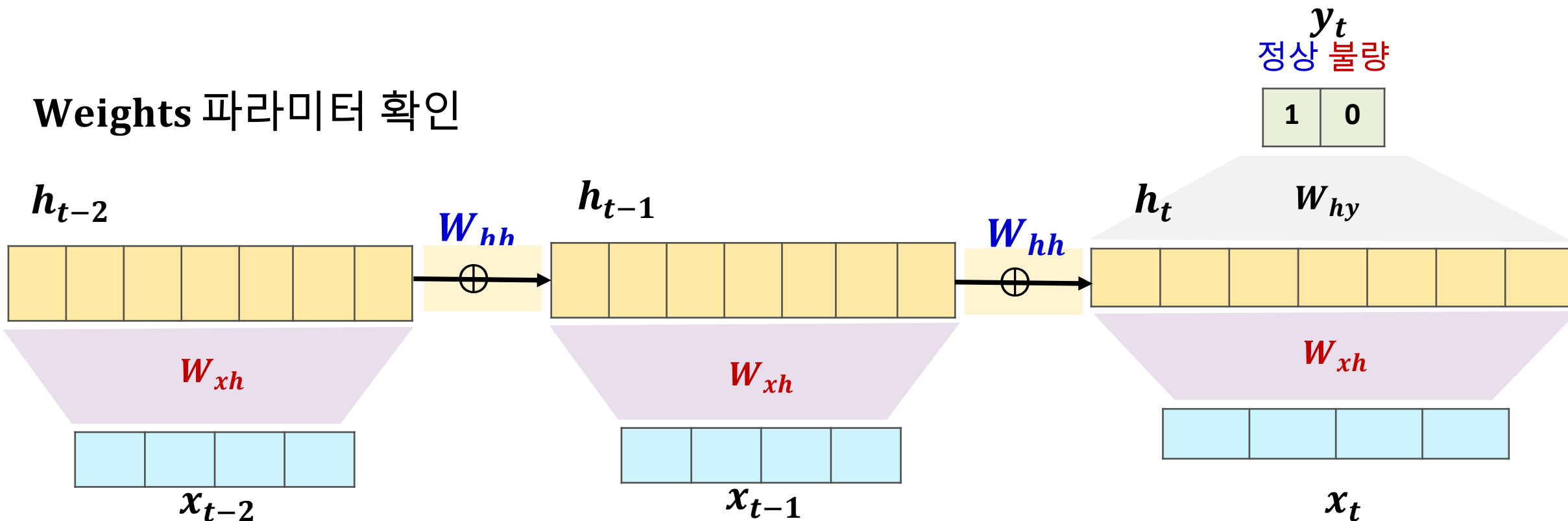
$$h_{t-1} = f(W_{xh} x_{t-1} + W_{hh} h_{t-2})$$

$$t-2 \quad h_t = f(W_{xh} x_t + W_{hh} h_{t-1})$$

$$t-1 \quad y_t = g(W_{hy} h_t)$$

$$t \quad f(\cdot) = \tanh \quad g(\cdot) = \text{softmax}$$

Weights 파라미터 확인



순환신경망(Recurrent Neural Network) 필요성

시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량

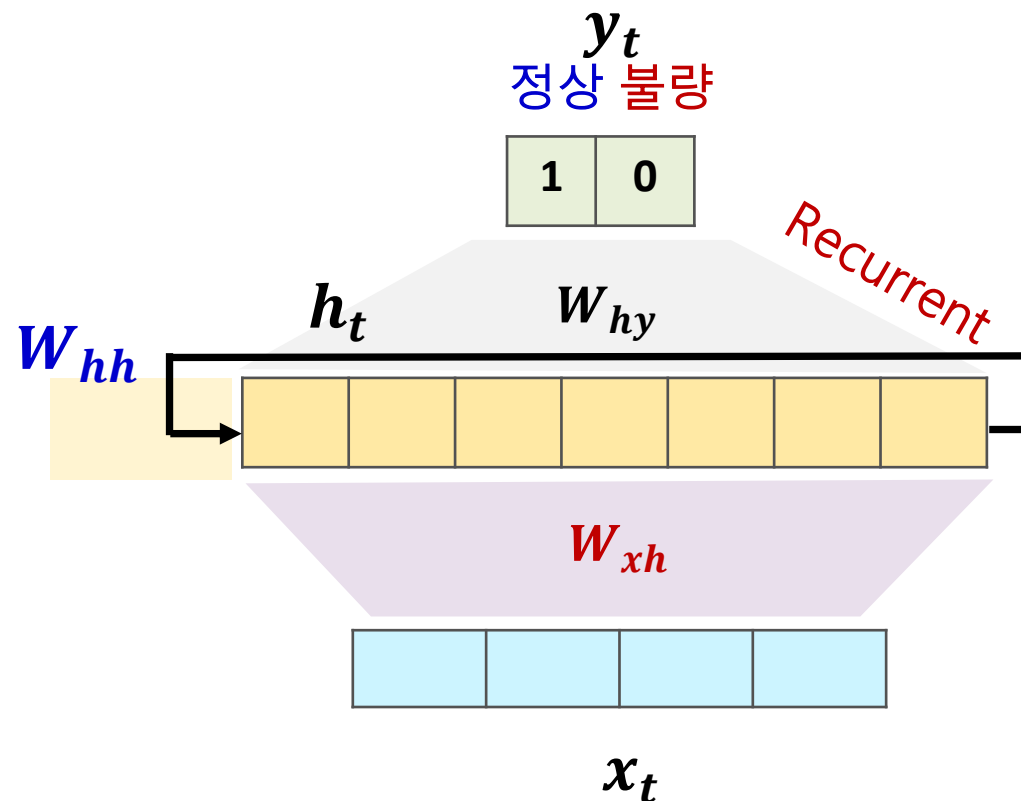
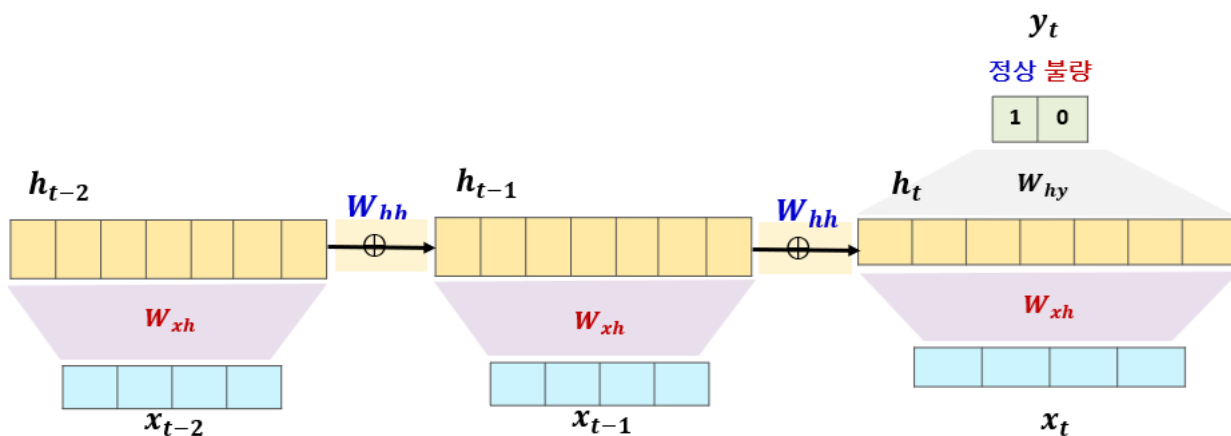
$$h_{t-1} = f(W_{xh} x_{t-1} + W_{hh} h_{t-2})$$

$$t-2 \quad h_t = f(W_{xh} x_t + W_{hh} h_{t-1})$$

$$t-1 \quad y_t = g(W_{hy} h_t)$$

$$t \quad f(\cdot) = \tanh \quad g(\cdot) = \text{softmax}$$

RNN은 이전 정보들이 순환(반복)하여 입력

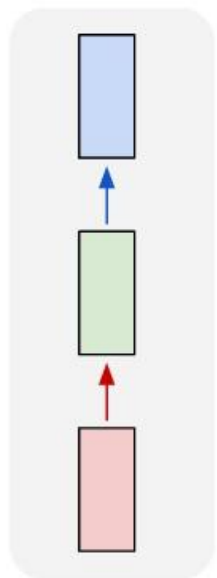


순환신경망 구조 다양성

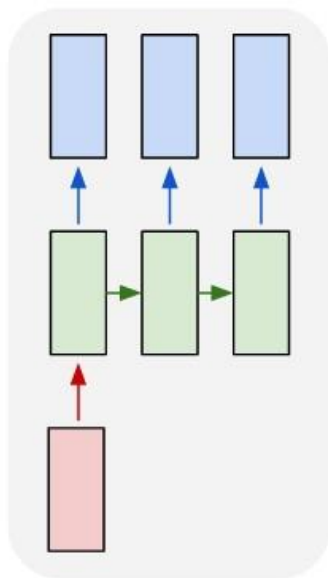
순환 신경망 구조의 종류

- 순차적으로 입력하고, 순차적으로 예측하는 것이 가능
- 순차적인 입력의 길이, 순차적인 예측의 길이에 따라 다음과 같이 구분됨

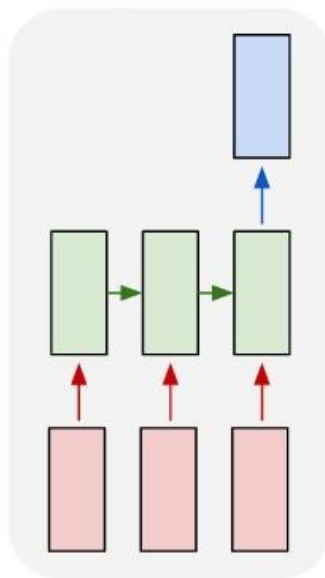
one to one



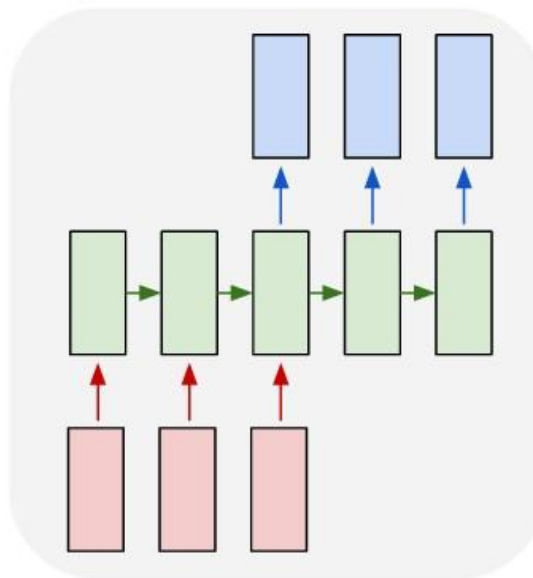
one to many



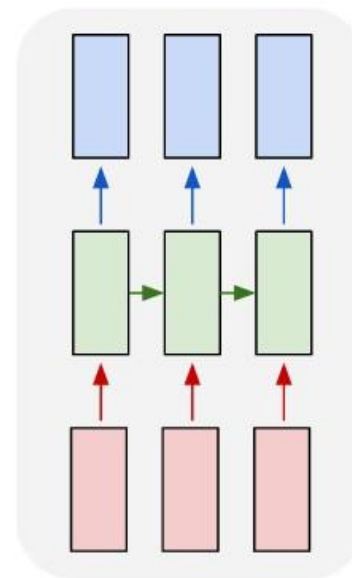
many to one



many to many



many to many

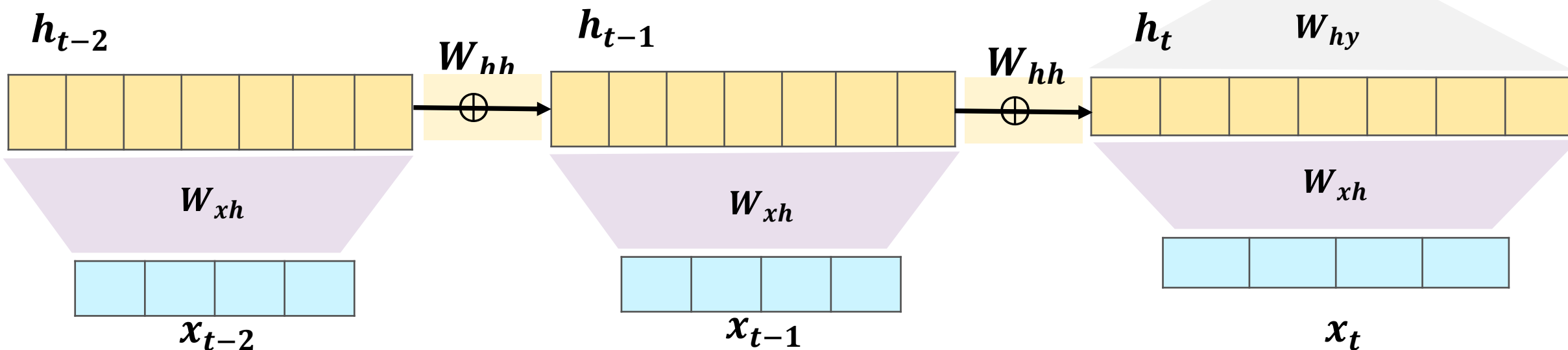
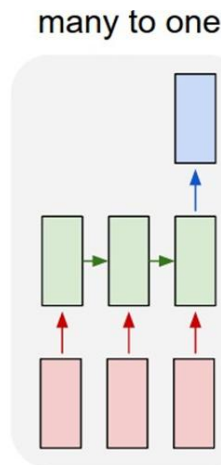


순환신경망 구조 다양성

Many to one 구조

- 여러시점(many) X로 하나의(one) Y를 예측하는 문제
- 예시 : 여러 시점에 다변화 센서 데이터가 주어졌을 때 특정 시점의 제품 상태를 예측

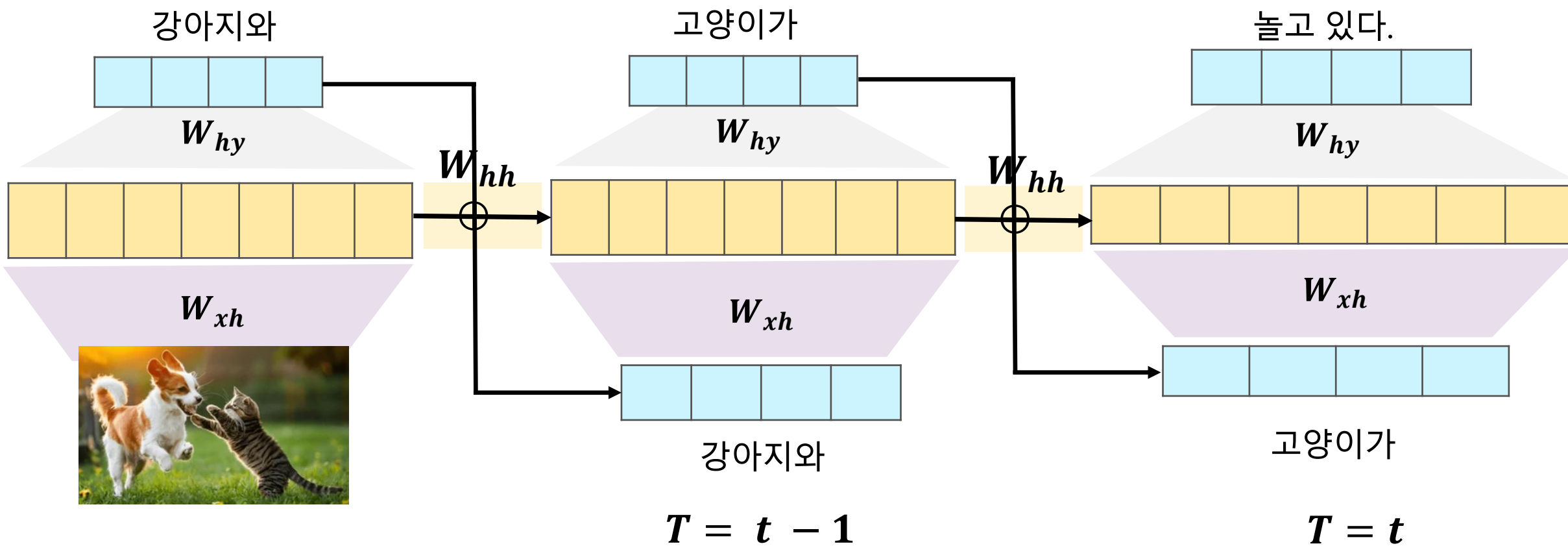
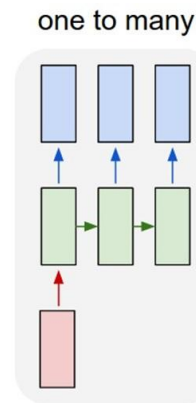
시간	센서1	센서2	센서3	상태
12:00	0	98.9	3.9	정상
13:00	0	81.9	3.9	정상
14:00	-0.3	74.5	6.7	불량



순환신경망 구조 다양성

One to many 구조

- 단일 시점(one) X로 순차적인(many) Y를 예측하는 문제
- 예시 : 이미지 데이터가 주어졌을 때, 이미지에 대한 정보를 글로 생성하는 이미지 캡셔닝

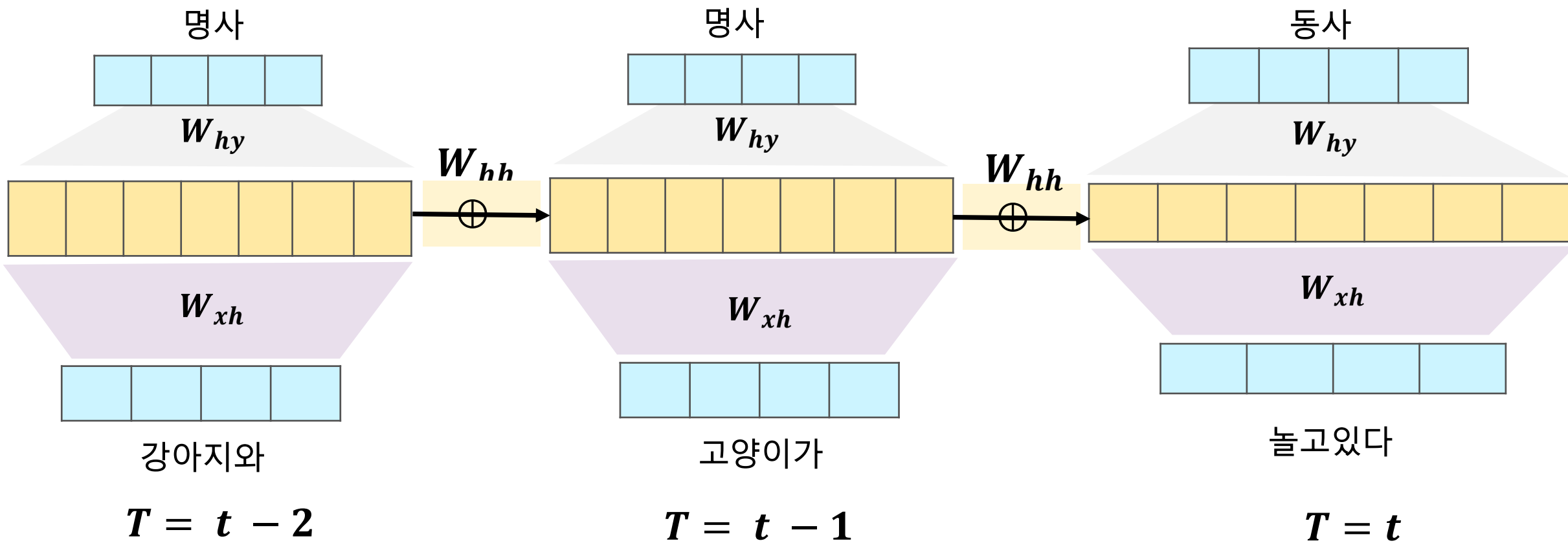
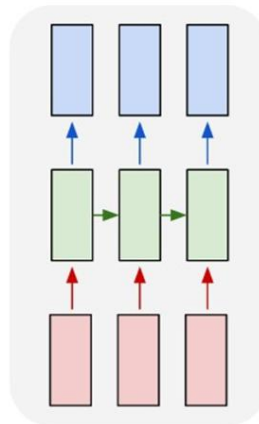


순환신경망 구조 다양성

Many to many 구조

- 순차적인(many) X로 순차적인(many) Y를 예측하는 문제
- 예시 : 문장이 주어졌을 때, 각 단어의 품사를 예측하는 Pos Tagging

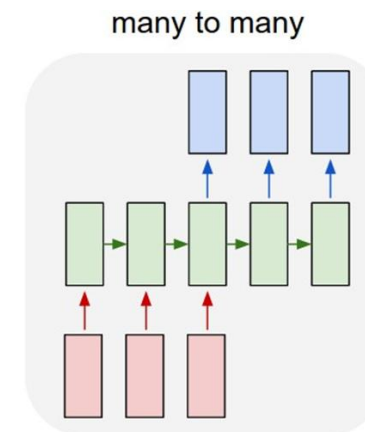
many to many



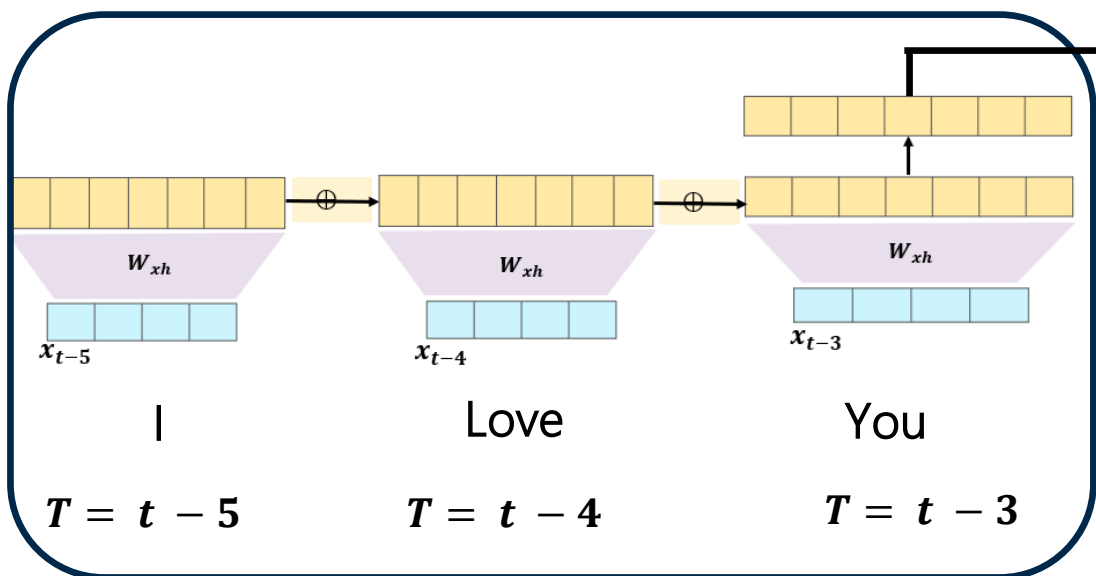
순환신경망 구조 다양성

Many to many 구조 (many to one + one to many)

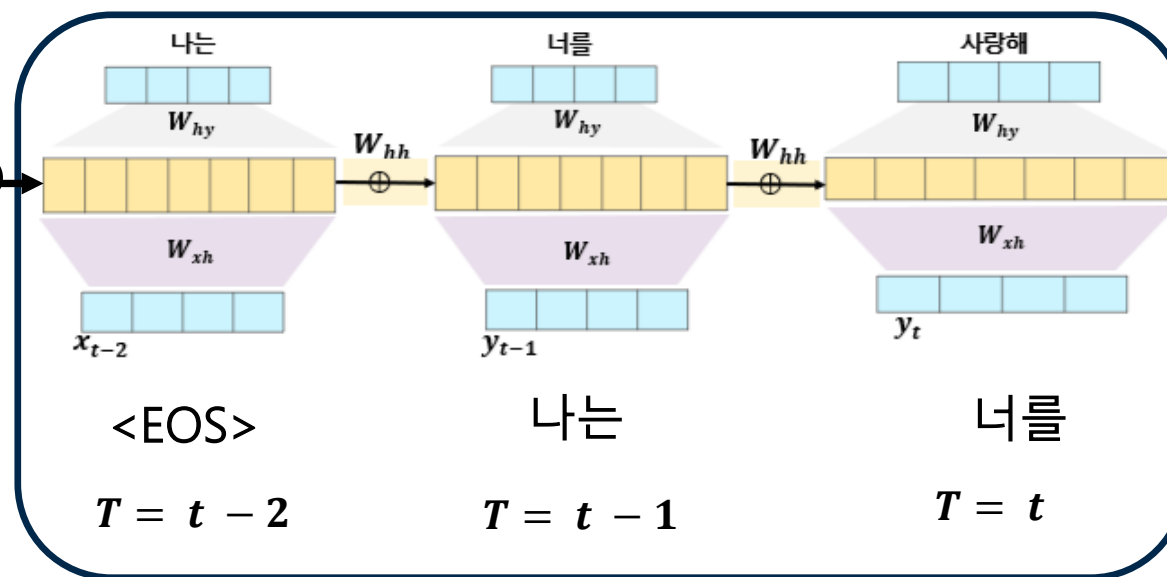
- 순차적인(many) X로 순차적인(many) Y를 예측하는 문제
- 예시 : 영어 문장을 한글 문장으로 번역



Many to one



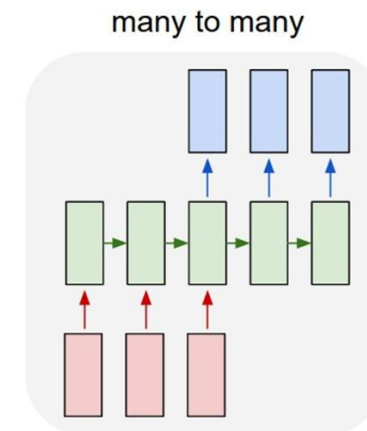
One to many



순환신경망 구조 다양성

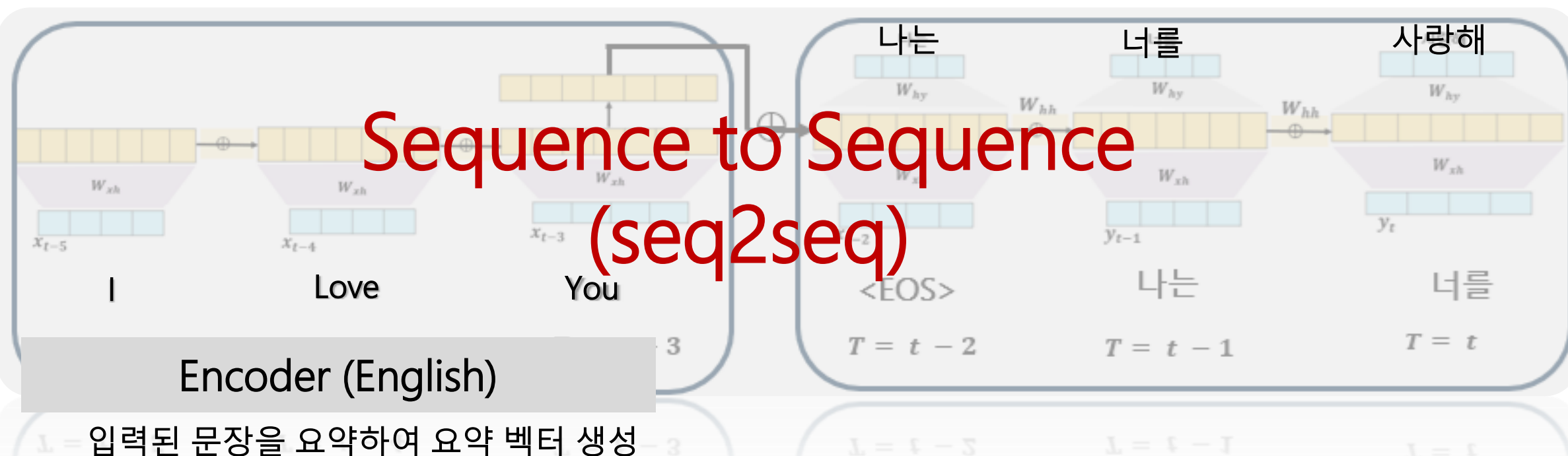
Many to many 구조 (many to one + one to many)

- 순차적인(many) X로 순차적인(many) Y를 예측하는 문제
- 예시 : 영어 문장을 한글 문장으로 번역



요약 벡터를 기반으로 출력 단어들을 예측

Decoder (Korean)

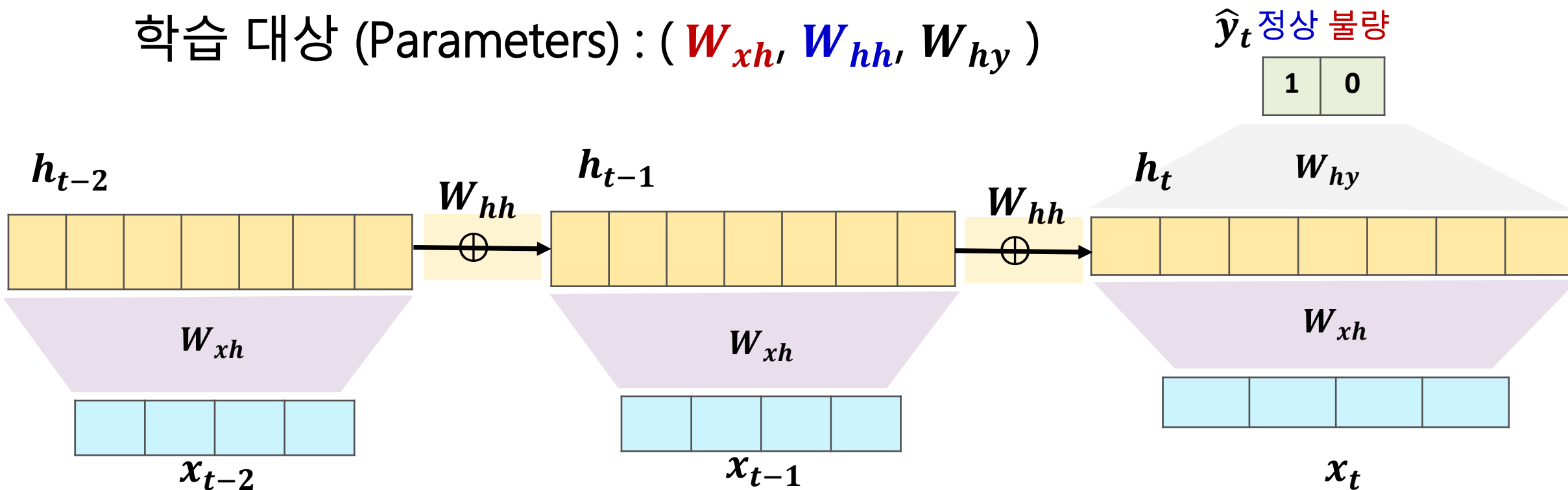


순환신경망 학습

순환 신경망은 t 시점까지의 과거 정보를 활용하여 y_t 를 예측

- t 시점 데이터 반영 $\rightarrow W_{xh}$
- t 시점 이전 정보 반영 $\rightarrow W_{hh}$
- t 시점의 y 를 예측 $\rightarrow W_{hy}$

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})



순환신경망 학습

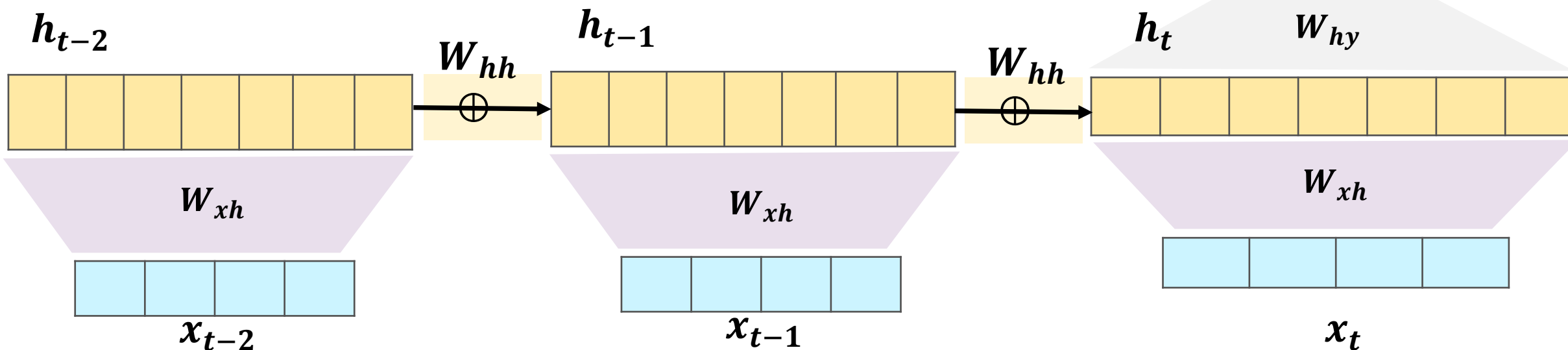
학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})

- 해당 파라미터는 매 시점마다 공유하는 구조 (Parameter sharing)
- 매 시점 파라미터를 구성하는 값이 같음
- 최적의 W : W 를 매 시점 적용했을 때 **Loss**가 최소가 되도록 **정상 불량**

$$h_t = f(W_{xh} x_t + W_{hh} h_{t-1})$$
$$\hat{y}_t = g(W_{hy} h_t)$$

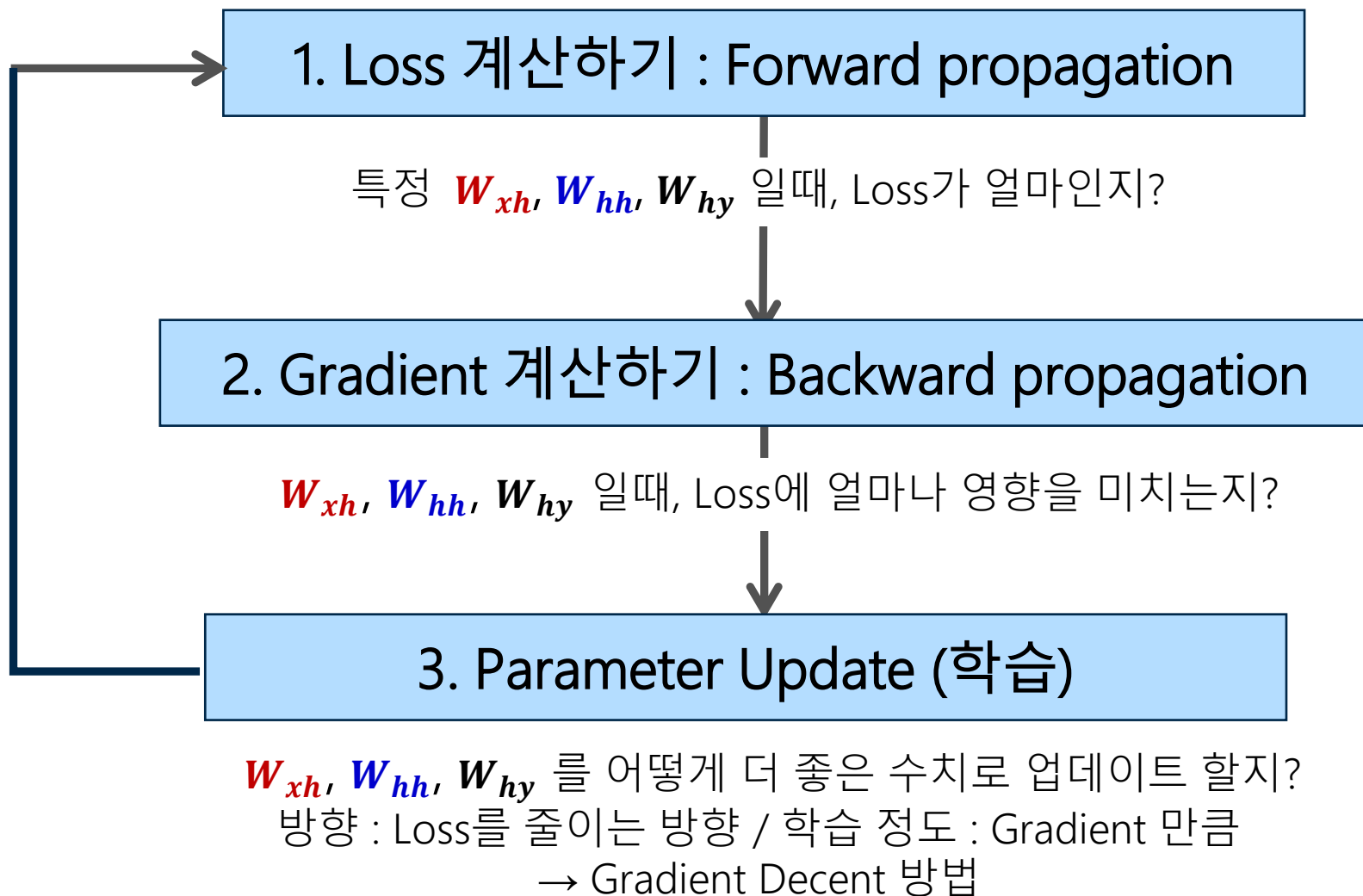
y_t	1	0
	정상	불량
$\hat{y}_t$	0.1	0.9
	정상	불량

Loss



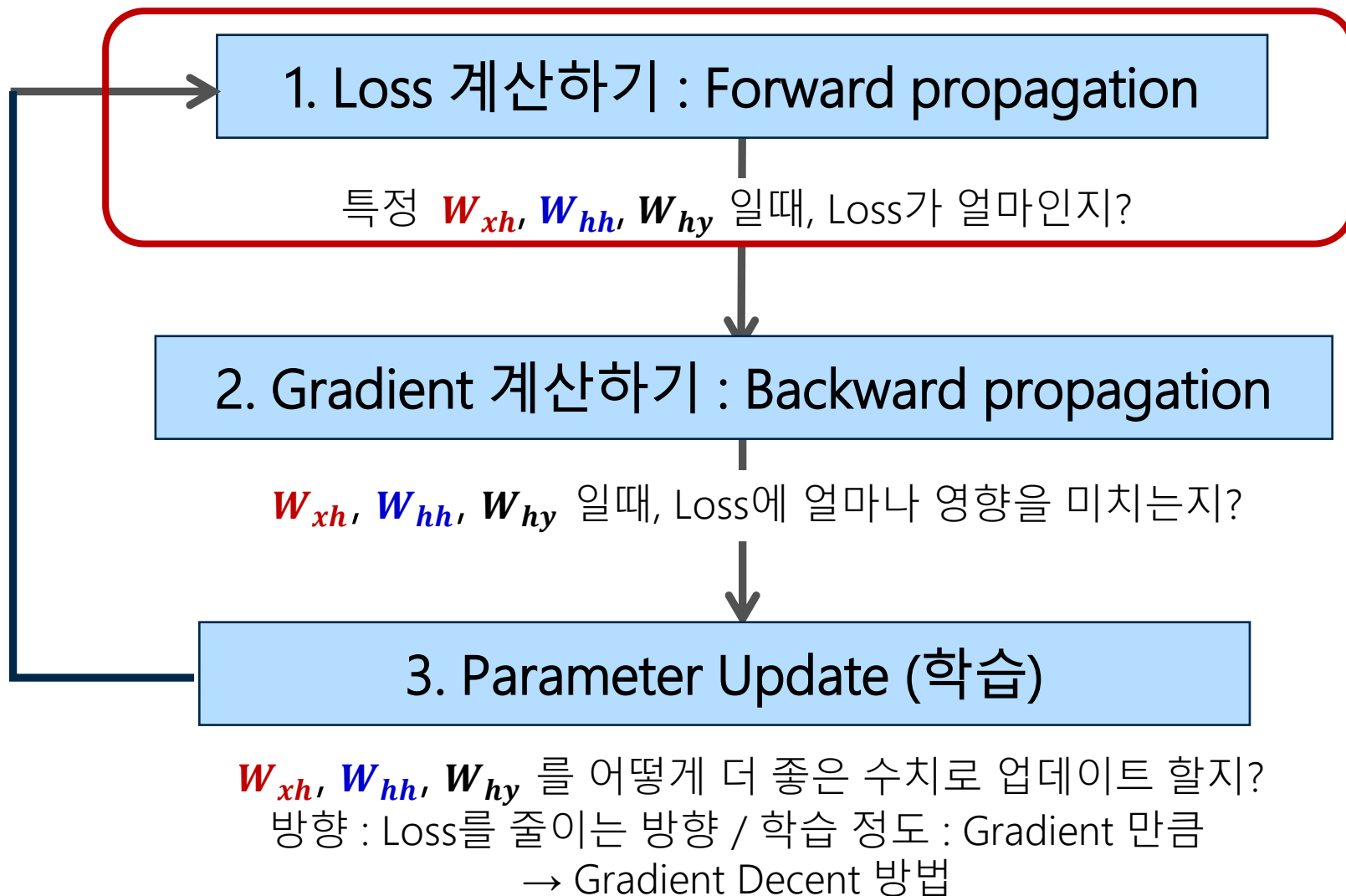
순환신경망 학습

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})



순환신경망 학습

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})

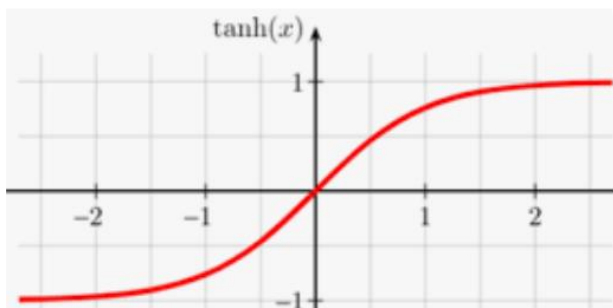


순환신경망 학습

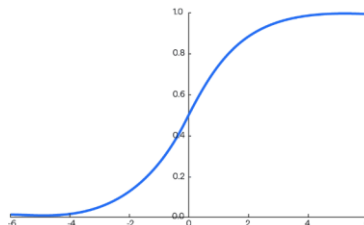
1. Loss 계산하기 : Forward propagation

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})

- $h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$: 각 weight 곱하고 더한 값을 -1 ~ 1사이로 정규화 한 hidden state
- $\hat{y}_t = \text{softmax}(W_{hy} h_t)$: Hidden state에 weight를 곱하고 0~1 사이로 정규화한 예측값
- $L_t = y_t$ 와 $\hat{y}_t$ 차이

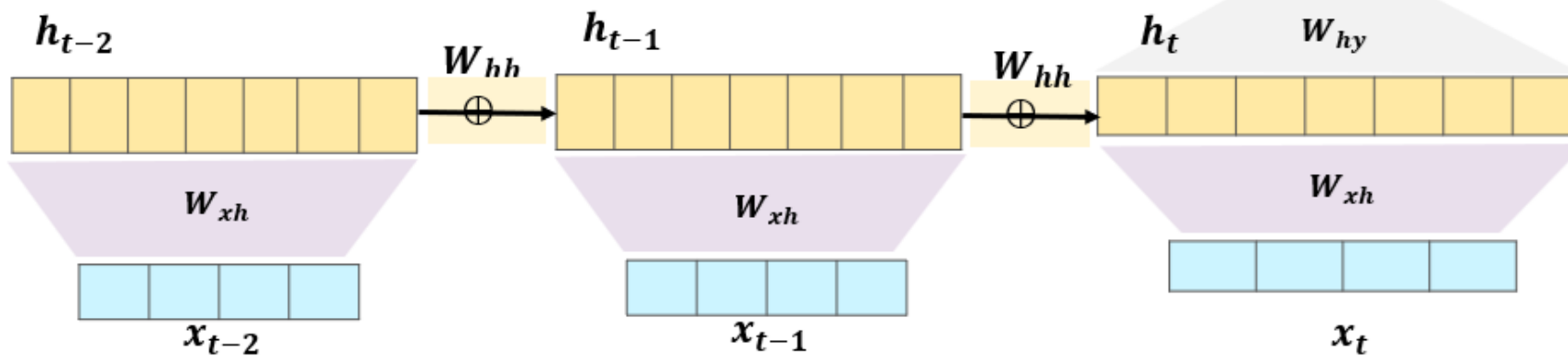


Softmax Function



	정상	불량
y_t	1	0
	정상	불량
$\hat{y}_t$	0.1	0.9

Loss



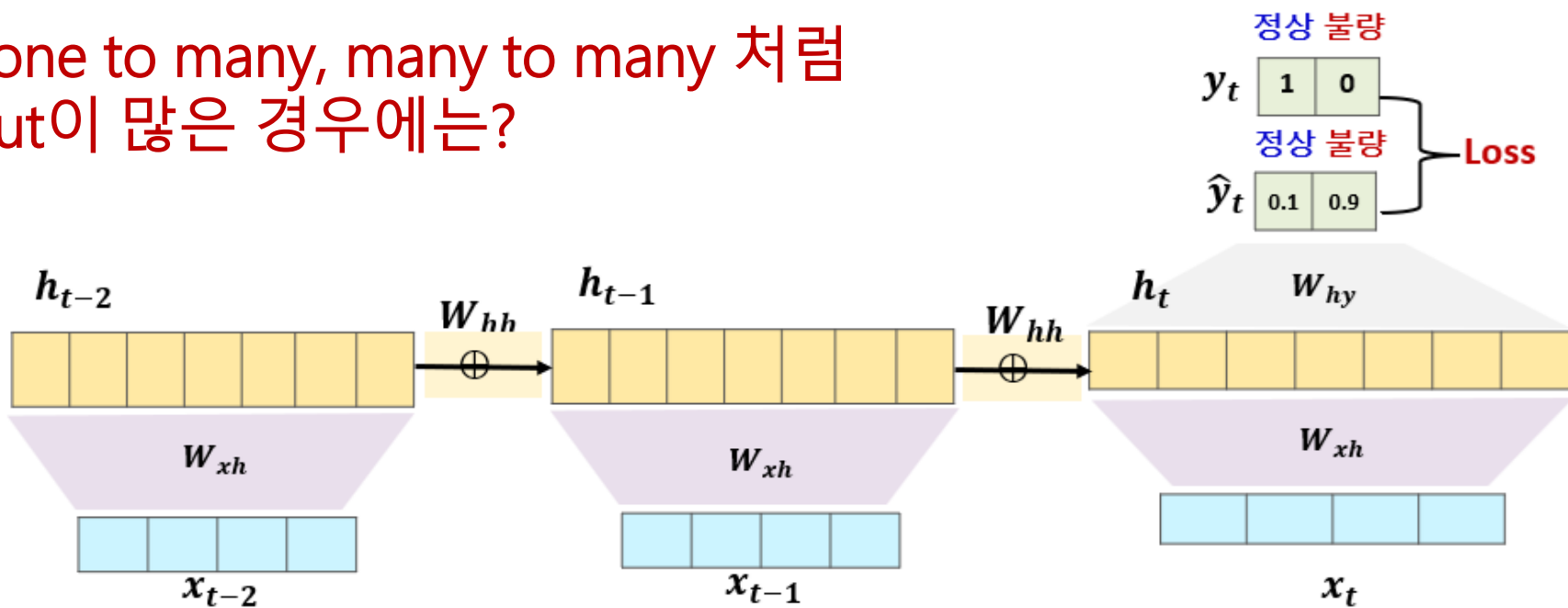
순환신경망 학습

1. Loss 계산하기 : Forward propagation

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})

- $h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$: 각 weight 곱하고 더한 값을 -1 ~ 1사이로 정규화 한 hidden state
- $\hat{y}_t = \text{softmax}(W_{hy} h_t)$: Hidden state에 weight를 곱하고 0~1 사이로 정규화한 예측값
- $L_t = y_t$ 와 $\hat{y}_t$ 차이

만약 one to many, many to many 처럼
output이 많은 경우에는?

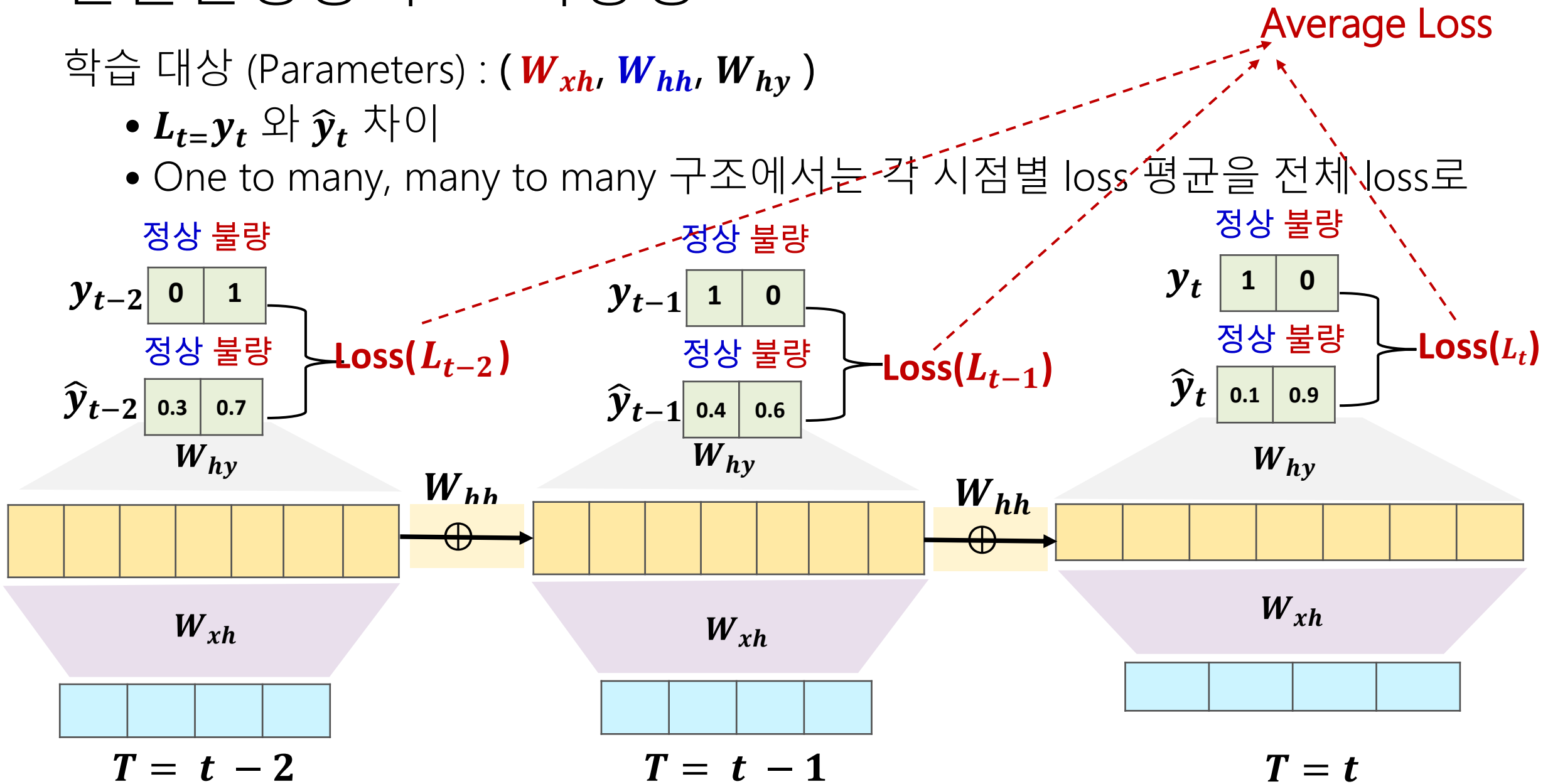


순환신경망 구조 다양성

1. Loss 계산하기 : Forward propagation

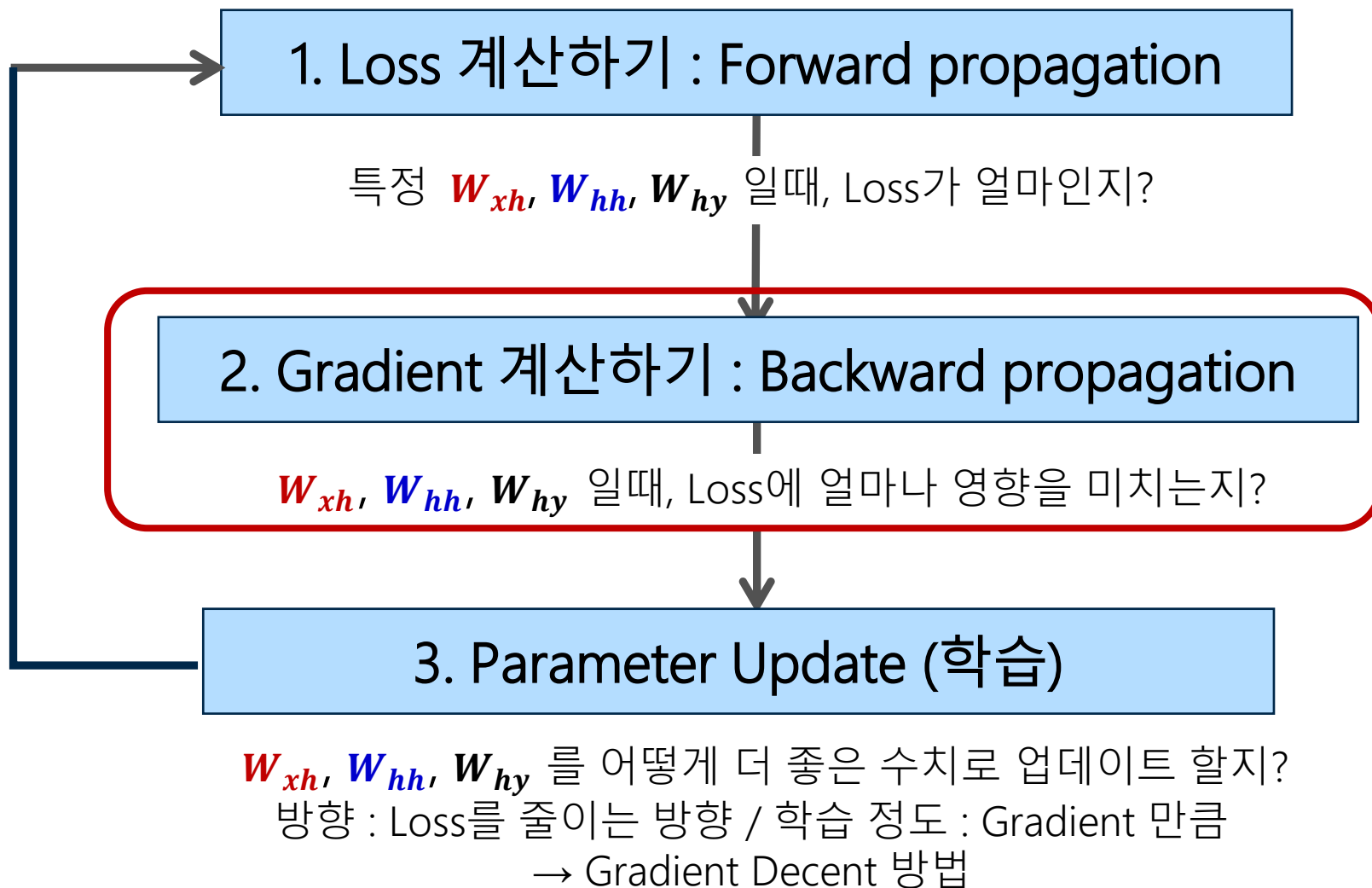
학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})

- $L_t = y_t$ 와 $\hat{y}_t$ 차이
- One to many, many to many 구조에서는 각 시점별 loss 평균을 전체 loss로



순환신경망 학습

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})

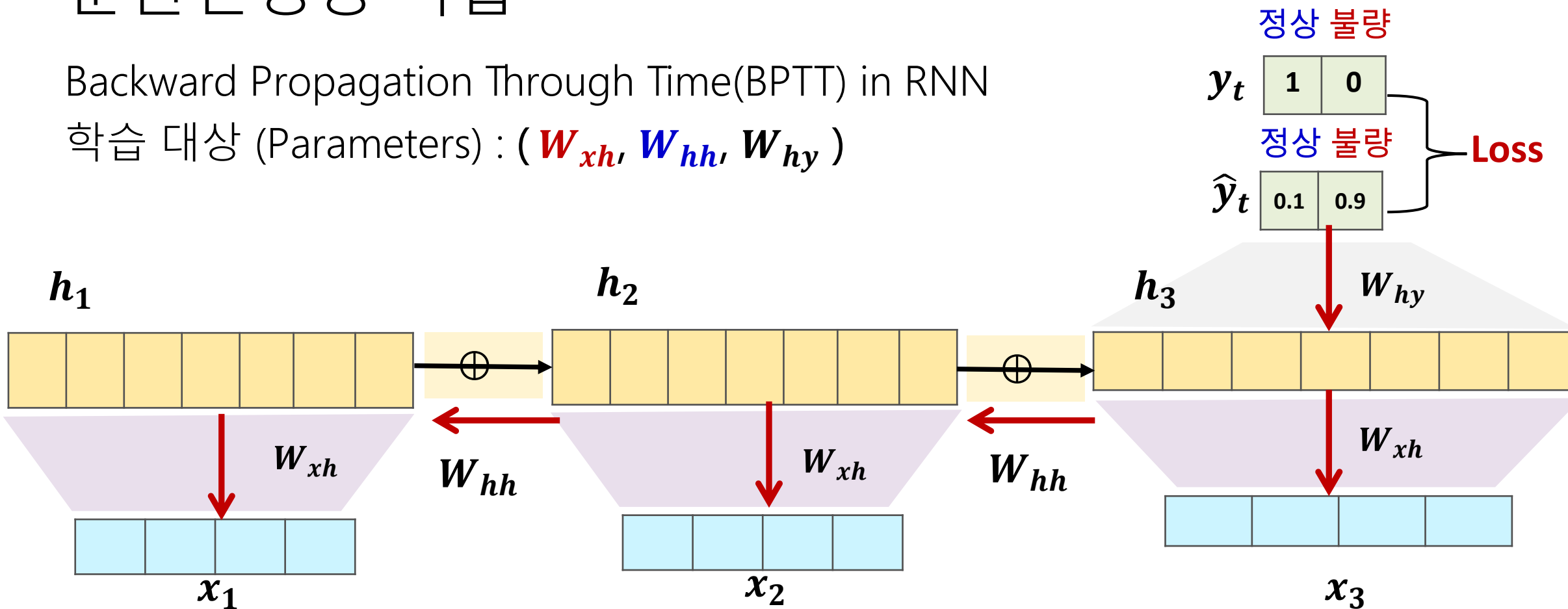


순환신경망 학습

2. Gradient 계산하기 : Backward propagation

Backward Propagation Through Time(BPTT) in RNN

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})



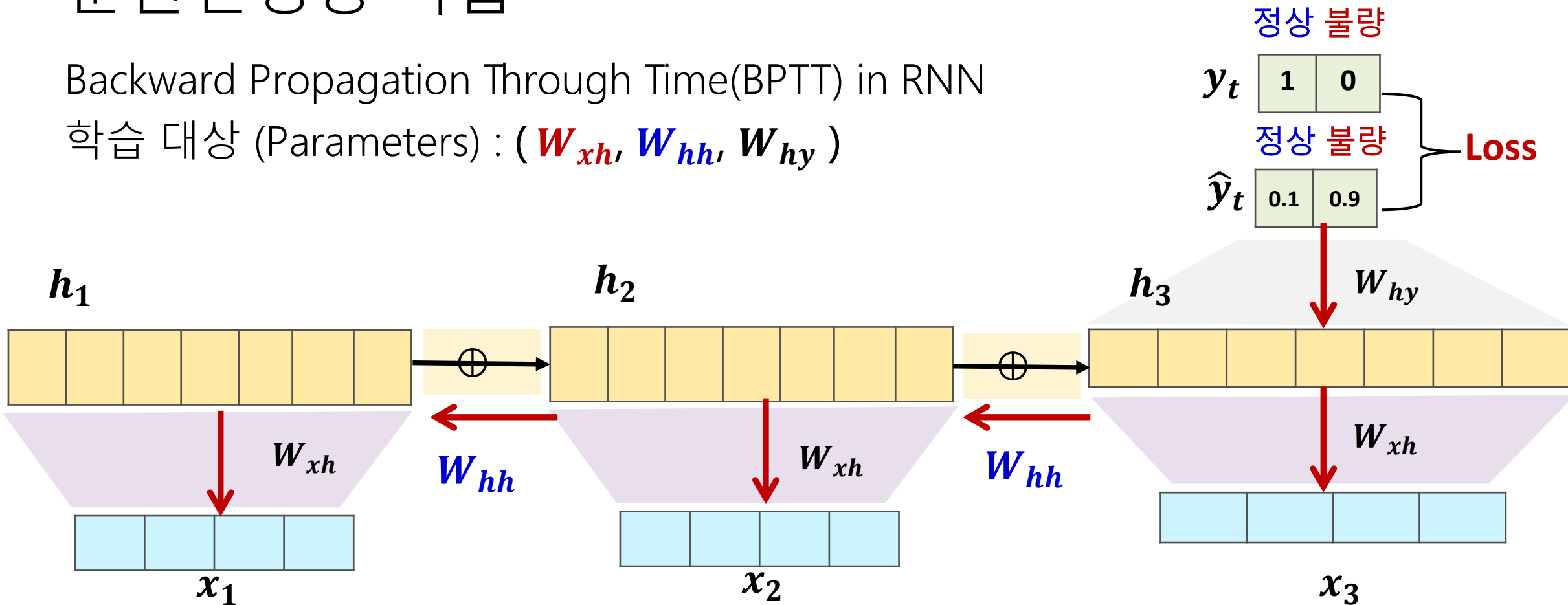
$$\frac{\partial Loss}{\partial W_{hy}} = \frac{\partial L_t}{\partial W_{hy}} = \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial W_{hy} h_t} \times \frac{\partial W_{hy} h_t}{\partial W_{hy}}$$

순환신경망 학습

2. Gradient 계산하기 : Backward propagation

Backward Propagation Through Time(BPTT) in RNN

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})



T_3 : 시점 3에서의 영향 T_2 : 시점 3으로부터 전해진 영향 고려 T_1 : 시점 2으로부터 전해진 영향 고려

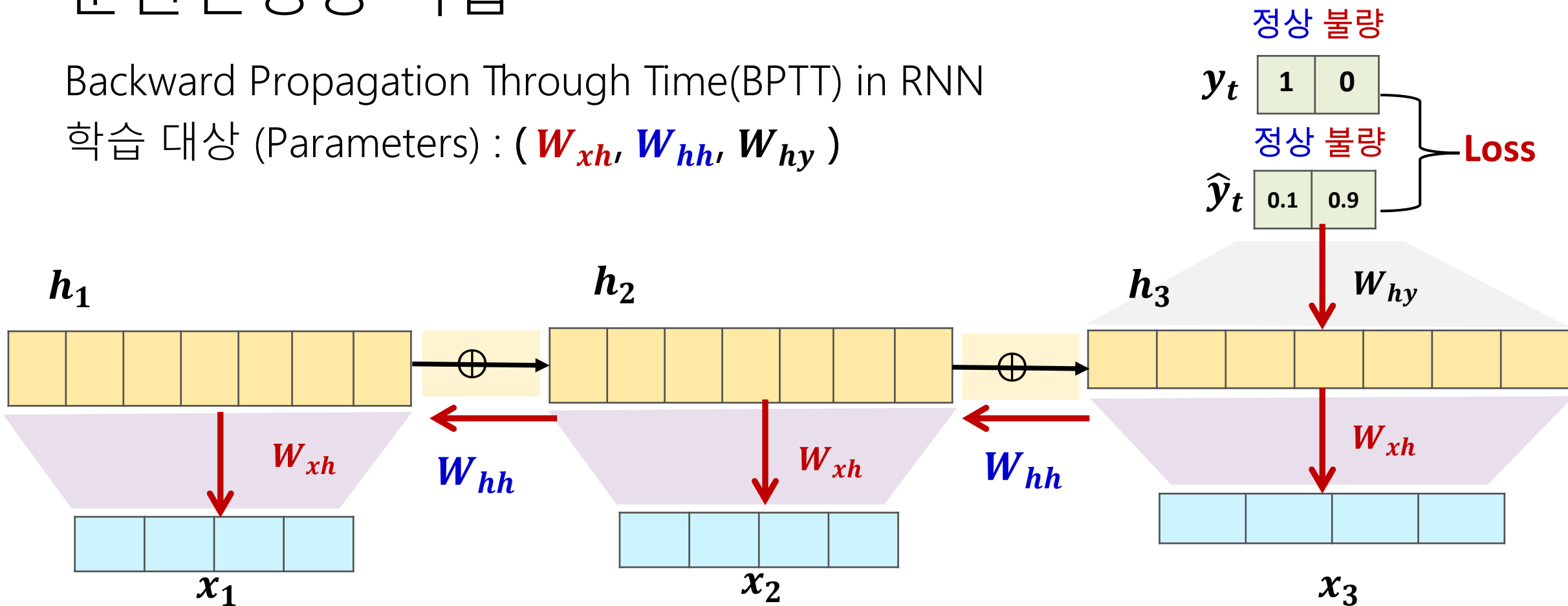
$$\frac{\partial Loss}{\partial W_{hh}} = \left[\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_3} \times \frac{\partial h_3}{\partial W_{hh}} \right] + \left[\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial W_{hh}} \right] + \left[\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial W_{hh}} \right]$$

순환신경망 학습

2. Gradient 계산하기 : Backward propagation

Backward Propagation Through Time(BPTT) in RNN

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})

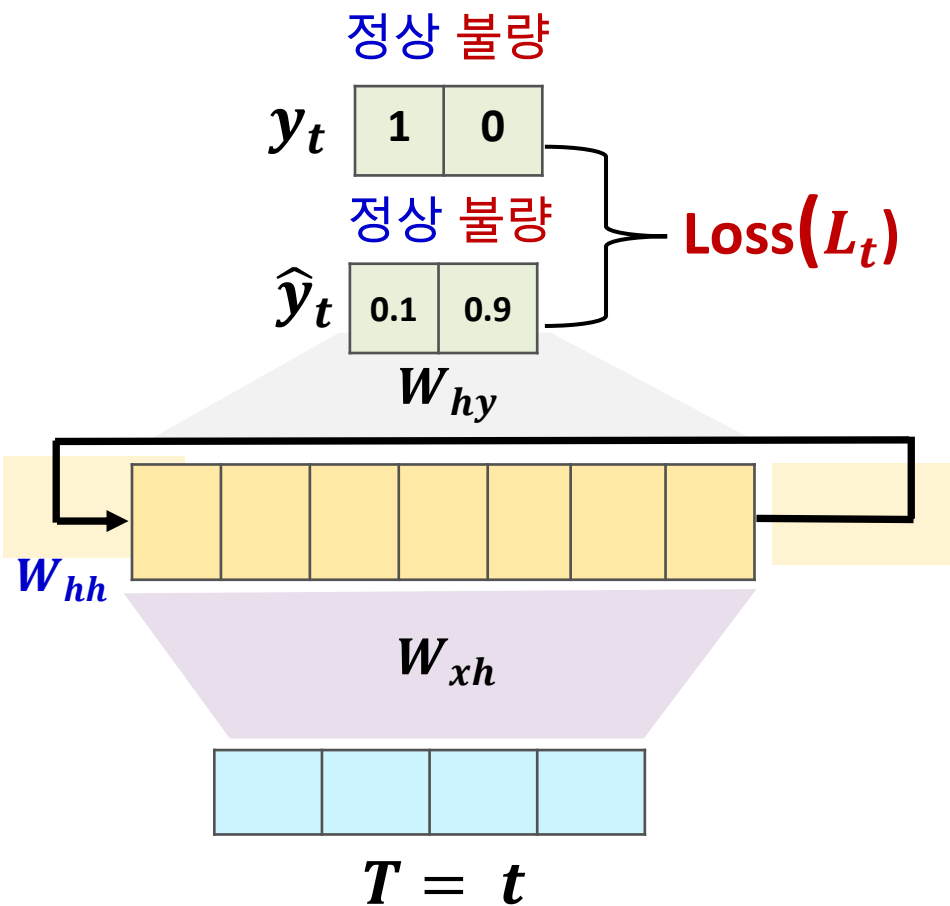


T_3 : 시점 3에서의 영향 T_2 : 시점 3으로부터 전해진 영향 고려 T_1 : 시점 2으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial W_{xh}} = \left[\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_3} \times \frac{\partial h_3}{\partial W_{xh}} \right] + \left[\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial W_{xh}} \right] + \left[\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial W_{xh}} \right]$$

순환신경망 학습

학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})



2. Gradient 계산하기 : Backward propagation

$$\frac{\partial Loss}{\partial W_{hy}} = \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial W_{hy} h_t} \times \frac{\partial W_{hy} h_t}{\partial W_{hy}}$$

$$\frac{\partial Loss}{\partial W_{hh}} = \sum_{k=0}^t \left(\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_t} \times \frac{\partial h_t}{\partial h_k} \times \frac{\partial h_k}{\partial W_{hh}} \right)$$

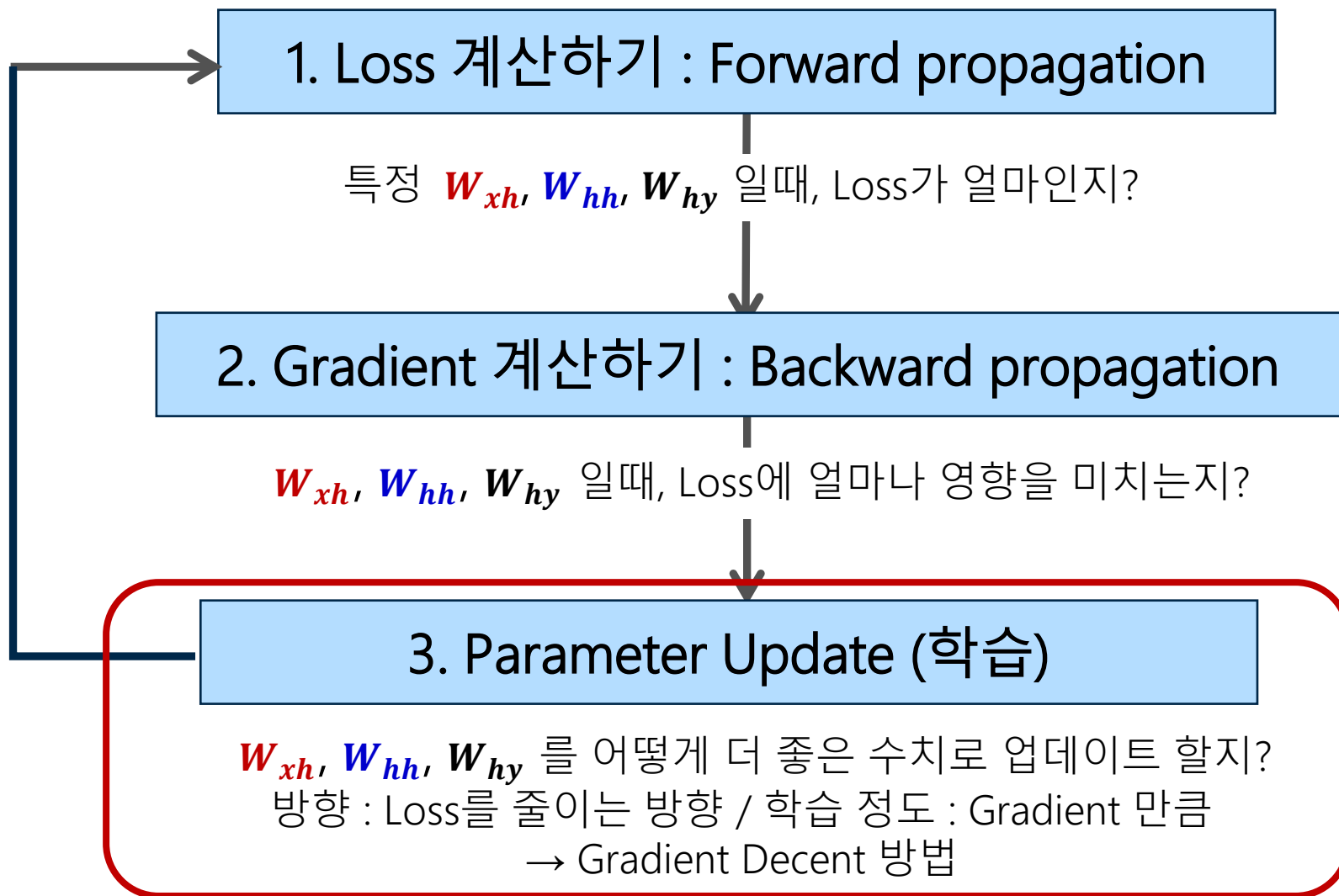
$$\frac{\partial Loss}{\partial W_{xh}} = \sum_{k=0}^t \left(\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_t} \times \frac{\partial h_t}{\partial h_k} \times \frac{\partial h_k}{\partial W_{xh}} \right)$$

회귀 Loss function(MSE) = $\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$

분류 Loss function(Cross Entropy) = $\sum_{i=1}^N y_{o,c} \log(P_{o,c})$

순환신경망 학습

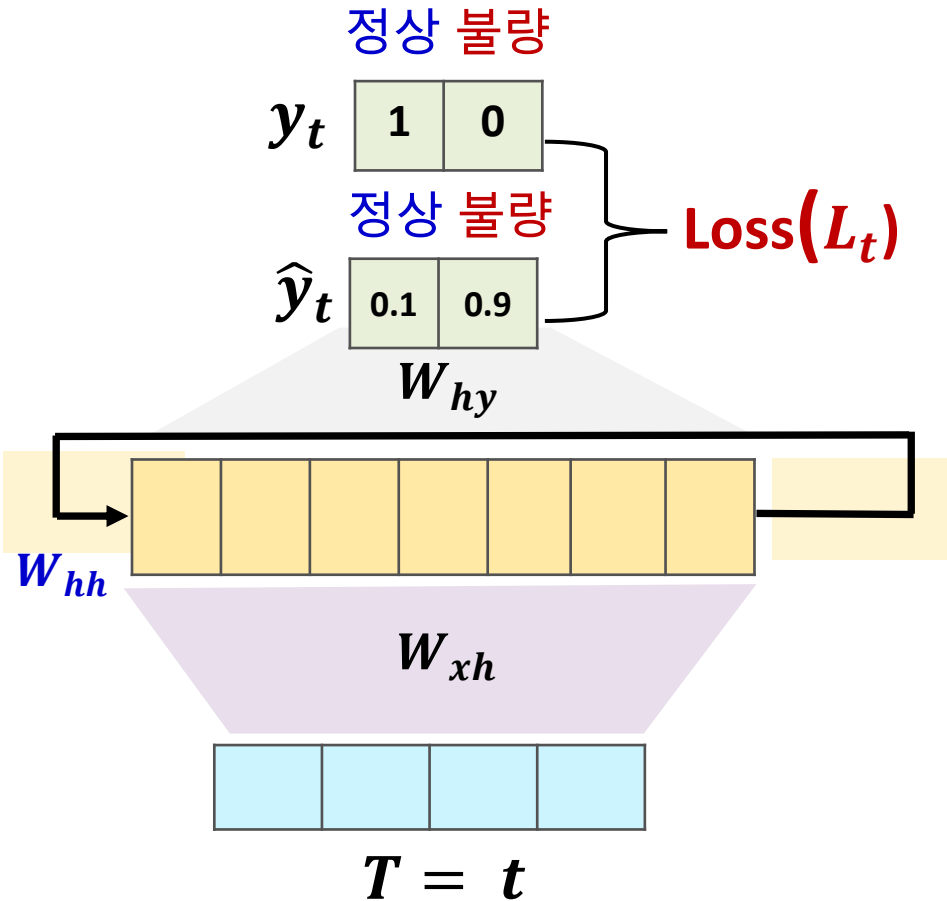
학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy})



순환신경망 학습

RNN에서의 parameter update

- 학습 대상 (Parameters) : (W_{xh} , W_{hh} , W_{hy}) → BPTT를 통해 gradient(기여도) 계산
- 학습률(η , learning rate) : 기여도를 얼마나 반영할 것인지 결정 (예: $\eta = 0.01$)



$$W_{hy} \text{ 기여도} = \frac{\partial Loss}{\partial W_{hy}}$$

$$\rightarrow W_{hy}^{new} = W_{hy}^{old} - \eta * \frac{\partial Loss}{\partial W_{hy}}$$

$$W_{hh} \text{ 기여도} = \frac{\partial Loss}{\partial W_{hh}}$$

$$\rightarrow W_{hh}^{new} = W_{hh}^{old} - \eta * \frac{\partial Loss}{\partial W_{hh}}$$

$$W_{xh} \text{ 기여도} = \frac{\partial Loss}{\partial W_{xh}}$$

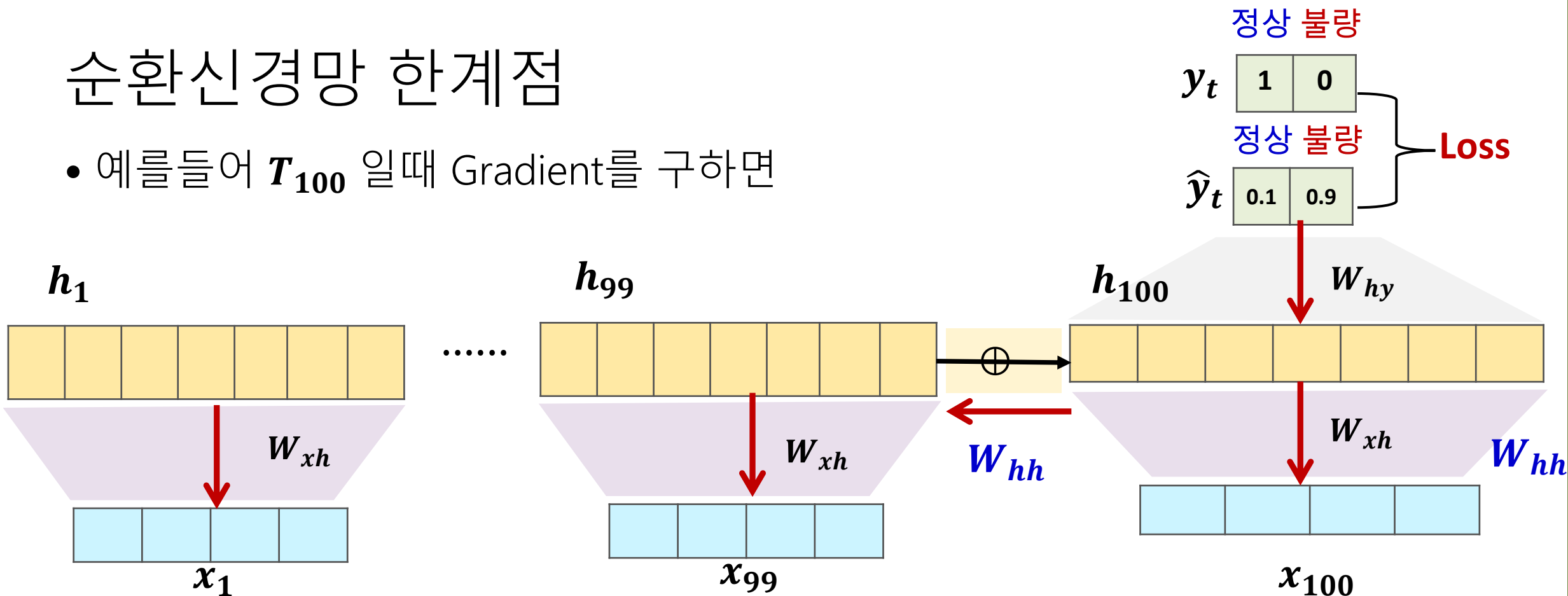
$$\rightarrow W_{xh}^{new} = W_{xh}^{old} - \eta * \frac{\partial Loss}{\partial W_{xh}}$$

순환신경망 한계점

- 장기 의존성 문제 (long-term dependency problem)
- Sequence의 길이가 길어질수록, 과거 정보 학습에 어려움이 발생
 - 원인 : 기울기 소실 (vanishing gradient)
 - Gradient는 chain rule에 의해 계산되며
 - 이때 $h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$

순환신경망 한계점

- 예를들어 T_{100} 일때 Gradient를 구하면



T_{100} : 시점 100에서의 영향

T_{99} : 시점 100으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial W_{hh}} = \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_{100}} \times \frac{\partial h_{100}}{\partial W_{hh}} + \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_{100}} \times \frac{\partial h_{100}}{\partial h_{99}} \times \frac{\partial h_{99}}{\partial W_{hh}} + \dots$$

$$+ \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_{100}} \times \frac{\partial h_{100}}{\partial h_{99}} \times \dots \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial W_{hh}}$$

T_1 : 시점 2으로부터 전해진 영향 고려

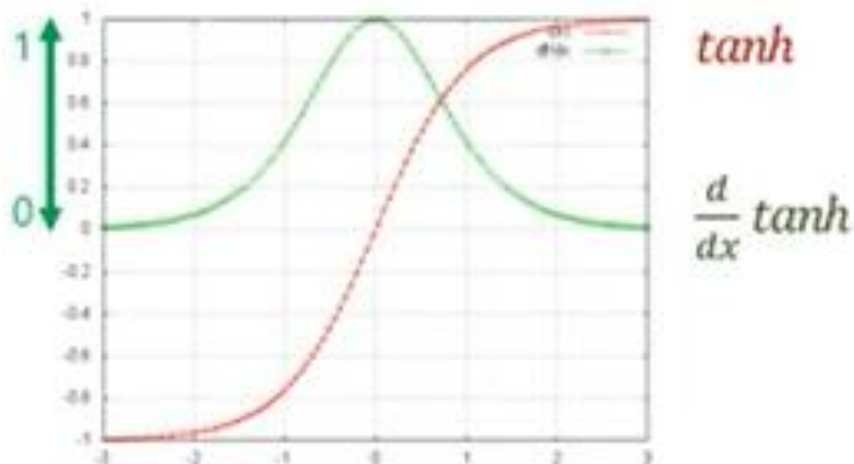
순환신경망 한계점

- 장기 의존성 문제 (long-term dependency problem)
- Sequence의 길이가 길어질수록, 과거 정보 학습에 어려움이 발생
 - 원인 : 기울기 소실 (vanishing gradient) → 장기 과거 시점에 대한 모델 학습이 안됨

$$W_{hh} \text{ 기여도} = \frac{\partial \text{Loss}}{\partial W_{hh}}$$

$$\rightarrow W_{hh}^{\text{new}} = W_{hh}^{\text{old}} - \eta * \frac{\partial \text{Loss}}{\partial W_{hh}}$$

$$\rightarrow W_{hh}^{\text{new}} = W_{hh}^{\text{old}} - \eta * \left(\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_{100}} \times \frac{\partial h_{100}}{\partial W_{hh}} + \dots + \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_{100}} \times \frac{\partial h_{100}}{\partial h_{99}} \times \dots \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial W_{hh}} \right)$$



$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{xh} x_t + W_{hh} h_{t-1}) W_{hh}$$

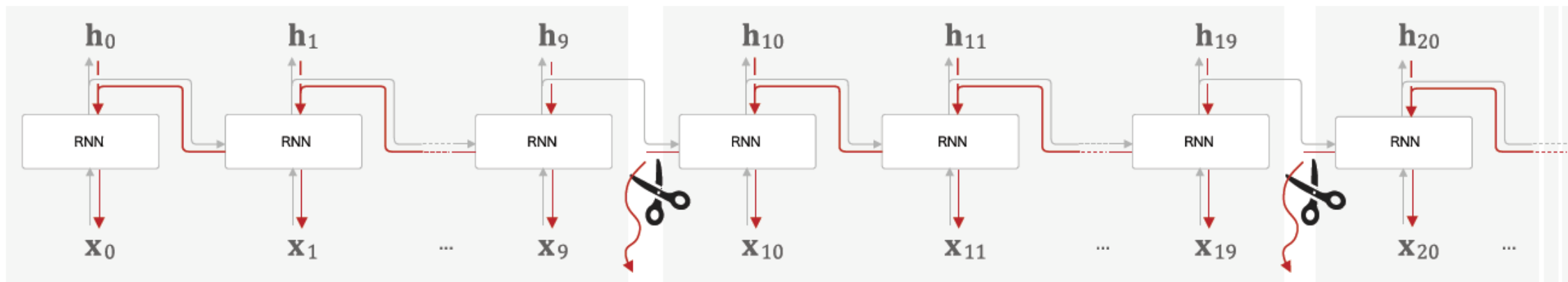
0부터 1사이의 값을 지님

주로 -1~1 사이 값

순환신경망 개선 모델 : Truncated BPTT

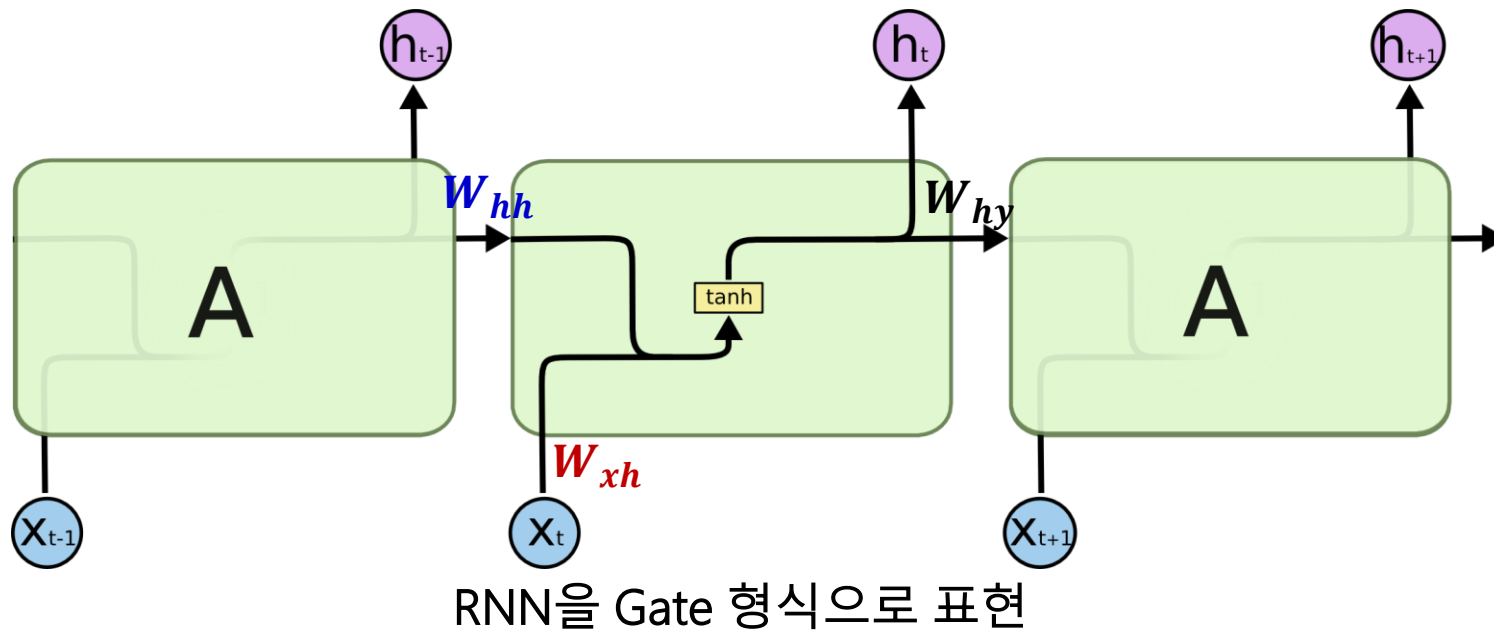
- 너무 길어진 신경망을 적당한 지점에서 잘라서 작은 여러 개의 신경망을 생성
- 순전파는 그대로 유지하고, 역전파에서만 끊어짐

역전파의 연결을 적당한 지점에서 끊는다. (역전파가 연결되는 일련의 RNN 계층을 '블록'이라 하고, 배경을 회색으로 칠해 다른 블록과 구분함)



순환신경망 개선 모델 : Long Short-Term Memory(LSTM)

- Long Short Term Memory(LSTM)은 장기 의존성 문제를 완화한 RNN 개선 모델
- Cell state (c_t)구조를 제안하고, 세가지 gate를 추가한 구조
 - 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)



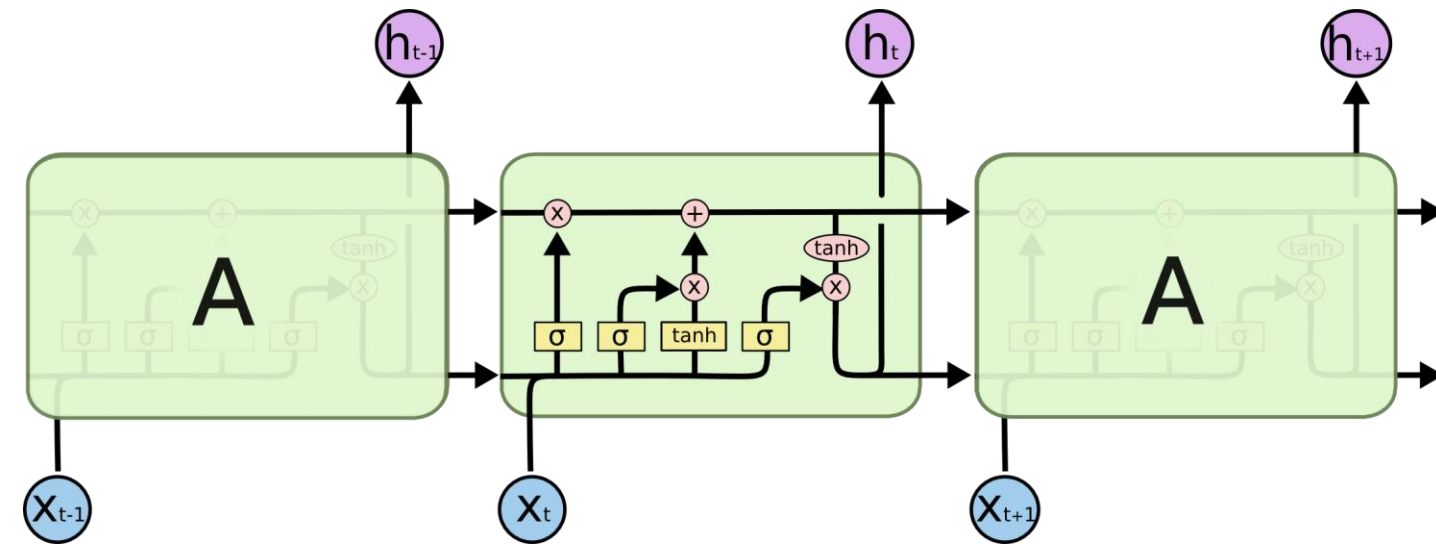
$$h_{t-1} = f(W_{xh} x_{t-1} + W_{hh} h_{t-2})$$

$$h_t = f(W_{xh} x_t + W_{hh} h_{t-1})$$

$$y_t = g(W_{hy} h_t)$$

순환신경망 개선 모델 : Long Short-Term Memory(LSTM)

- Long Short Term Memory(LSTM)은 장기 의존성 문제를 완화한 RNN 개선 모델
- Cell state (c_t)구조를 제안하고, 세가지 gate를 추가한 구조
- 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)

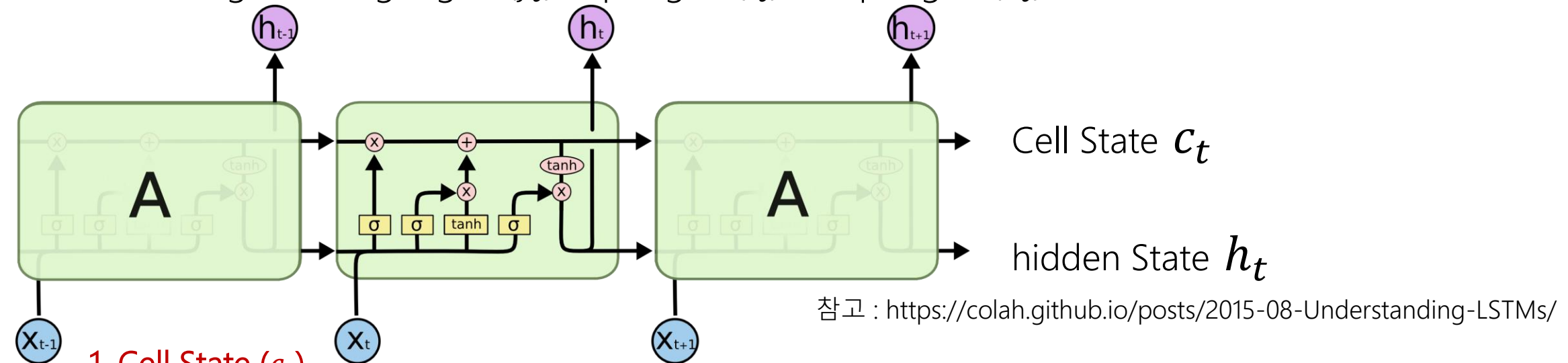


LSTM

$$\begin{aligned} f_t &= \sigma(W_{xhf} x_t + W_{hhf} h_{t-1} + b_{hf}) \\ i_t &= \sigma(W_{xhi} x_t + W_{hhi} h_{t-1} + b_{hi}) \\ o_t &= \sigma(W_{xho} x_t + W_{hho} h_{t-1} + b_{ho}) \\ \tilde{c}_t &= \tanh(W_{xhg} x_t + W_{hgg} h_{t-1} + b_{hg}) \\ c_t &= f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t \\ h_t &= o_t \otimes \tanh(c_t) \end{aligned}$$

순환신경망 개선 모델 : Long Short-Term Memory(LSTM)

- Long Short Term Memory(LSTM)은 장기 의존성 문제를 완화한 RNN 개선 모델
- Cell state (c_t)구조를 제안하고, 세가지 gate를 추가한 구조
- 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)



1. Cell State (c_t)

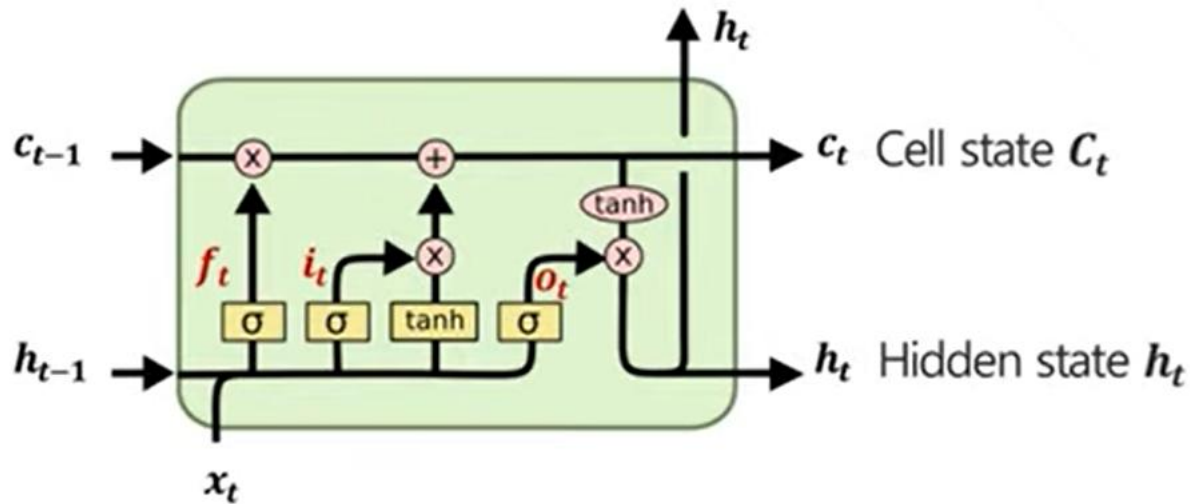
- 장기적으로 정보를 유지하는 장기 메모리(Long-term memory) 역할을 한다.
- 시간의 흐름에 따라 비교적 변형이 적게 일어나며, 필요한 정보만 게이트를 통해 추가하거나 삭제한다
- 시퀀스 전체에서 중요한 정보가 다음 시점까지 전달될 수 있도록 하는 정보 흐름의 주요 경로.

2. Hidden State (h_t)

- 현재 시점에서의 단기 메모리 역할을 하며, 직전 시점의 중요한 특징을 담아 출력으로 사용된다.
- 직전 시점의 정보를 포함하지만, 장기적 정보보다는 현재 시점의 상황을 반영하는 데 초점.
- 다음 시점 계산 시 입력으로 사용되며, 동시에 모델의 출력에도 직접 연결된다

순환신경망 개선 모델 : Long Short-Term Memory(LSTM)

- Long Short Term Memory(LSTM)은 장기 의존성 문제를 완화한 RNN 개선 모델
- Cell state (c_t)구조를 제안하고, 세가지 gate를 추가한 구조
- 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)



$$f_t = \sigma(W_{xhf} x_t + W_{hfh} h_{t-1} + b_{hf})$$

$$i_t = \sigma(W_{xhi} x_t + W_{hhi} h_{t-1} + b_{hi})$$

$$o_t = \sigma(W_{xho} x_t + W_{hho} h_{t-1} + b_{ho})$$

$$\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hgh} h_{t-1} + b_{hg})$$

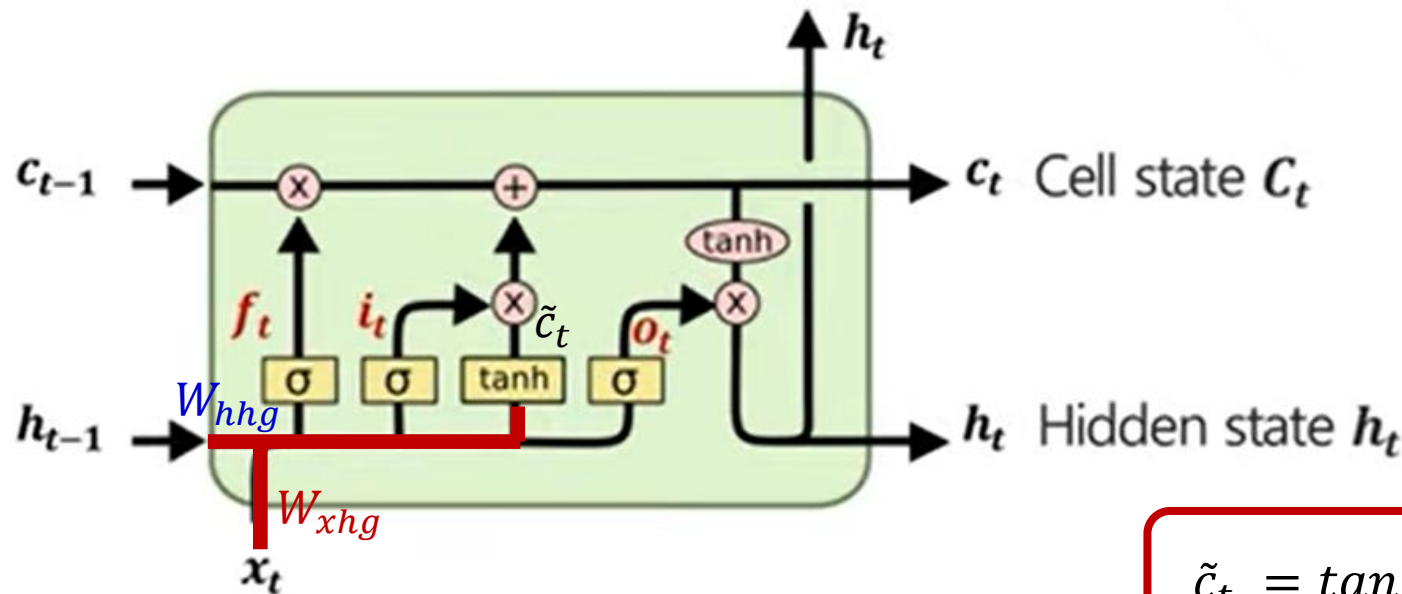
$$c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$$

$$h_t = o_t \otimes \tanh(c_t)$$

$$y_t = g(W_{hy} h_t)$$

순환신경망 개선 모델 : Long Short-Term Memory(LSTM)

- Long Short Term Memory(LSTM)은 장기 의존성 문제를 완화한 RNN 개선 모델
- Cell state (c_t)구조를 제안하고, 세가지 gate를 추가한 구조
- 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)



$$\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hhg} h_{t-1} + b_{hg})$$

$$f_t = \sigma(W_{xhf} x_t + W_{hhf} h_{t-1} + b_{hf})$$

$$i_t = \sigma(W_{xhi} x_t + W_{hhi} h_{t-1} + b_{hi})$$

$$o_t = \sigma(W_{xho} x_t + W_{hho} h_{t-1} + b_{ho})$$

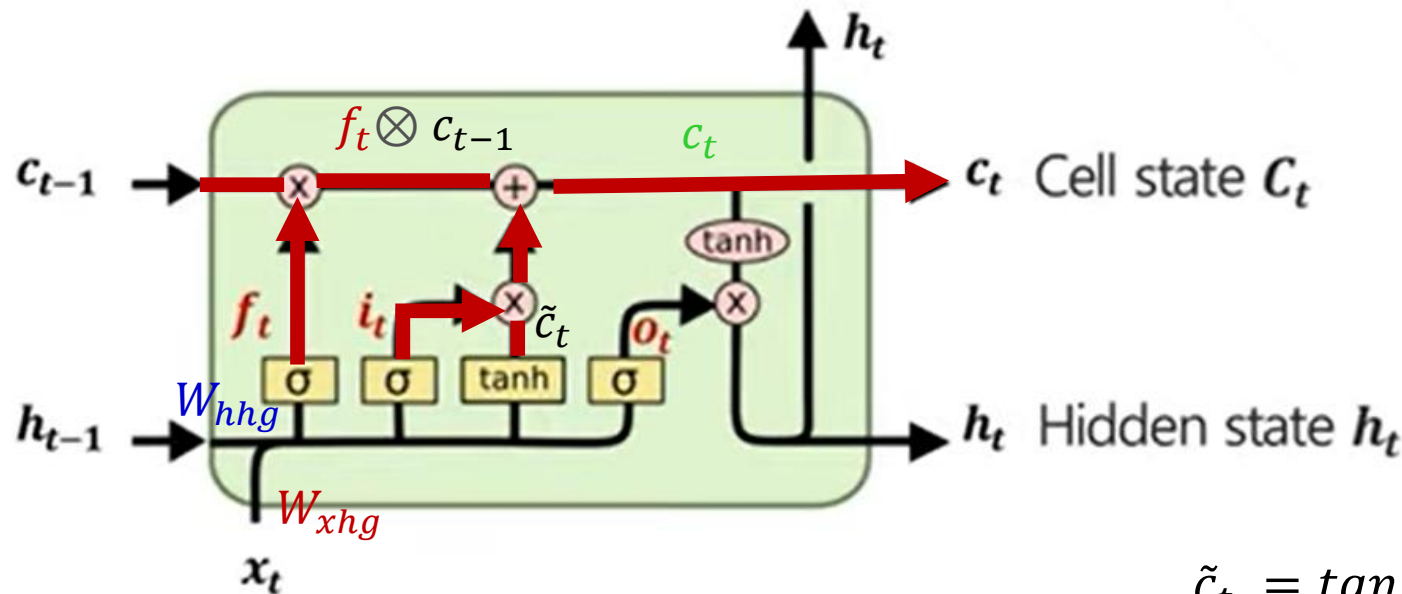
$$c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$$

$$h_t = o_t \otimes \tanh(c_t)$$

$$y_t = g(W_{hy} h_t)$$

순환신경망 개선 모델 : Long Short-Term Memory(LSTM)

- Long Short Term Memory(LSTM)은 장기 의존성 문제를 완화한 RNN 개선 모델
- Cell state (c_t)구조를 제안하고, 세가지 gate를 추가한 구조
- 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)



$$f_t = \sigma(W_{xhf} x_t + W_{hhf} h_{t-1} + b_{hf})$$

$$i_t = \sigma(W_{xhi} x_t + W_{hhi} h_{t-1} + b_{hi})$$

$$o_t = \sigma(W_{xho} x_t + W_{hho} h_{t-1} + b_{ho})$$

$$\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hhg} h_{t-1} + b_{hg})$$

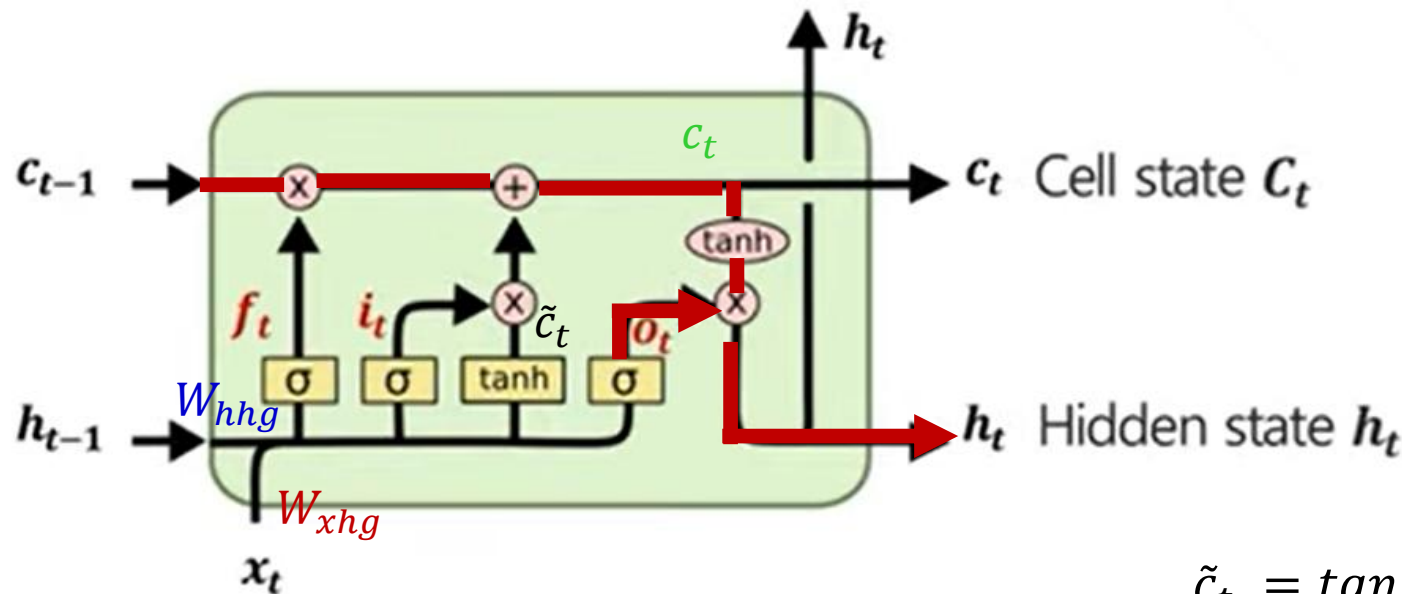
$$c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$$

$$h_t = o_t \otimes \tanh(c_t)$$

$$y_t = g(W_{hy} h_t)$$

순환신경망 개선 모델 : Long Short-Term Memory(LSTM)

- Long Short Term Memory(LSTM)은 장기 의존성 문제를 완화한 RNN 개선 모델
- Cell state (c_t)구조를 제안하고, 세가지 gate를 추가한 구조
- 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)



$$f_t = \sigma(W_{xhf} x_t + W_{hhf} h_{t-1} + b_{hf})$$

$$i_t = \sigma(W_{xhi} x_t + W_{hhi} h_{t-1} + b_{hi})$$

$$o_t = \sigma(W_{xho} x_t + W_{hho} h_{t-1} + b_{ho})$$

$$\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hhg} h_{t-1} + b_{hg})$$

$$c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$$

$$h_t = o_t \otimes \tanh(c_t)$$

$$y_t = g(W_{hy} h_t)$$

LSTM 구조

1. Gate 계산하기 : Forget (f_t), Input (i_t), gate(o_t)

$$f_t = \sigma(W_{xhf} x_t + W_{hhf} h_{t-1} + b_{hf})$$

$$i_t = \sigma(W_{xhi} x_t + W_{hhi} h_{t-1} + b_{hi})$$

$$o_t = \sigma(W_{xho} x_t + W_{hho} h_{t-1} + b_{ho})$$

2. Cell state (c_t) 업데이트 하기

$$\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hgg} h_{t-1} + b_{hg})$$

$$c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$$

3. Hidden state (h_t) 업데이트 하기

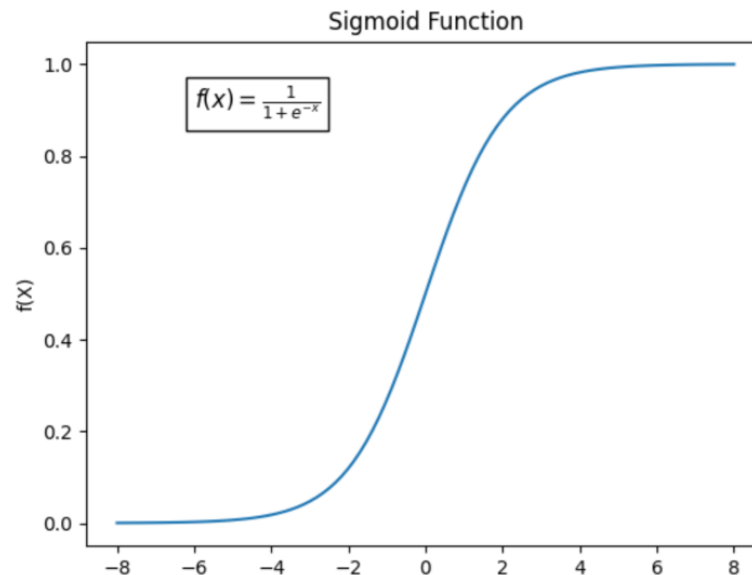
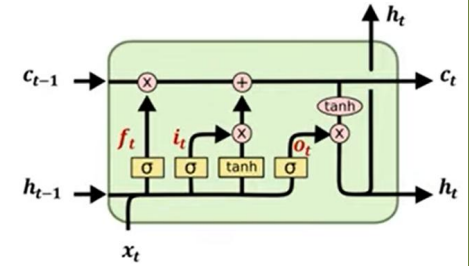
$$h_t = o_t \otimes \tanh(c_t)$$

LSTM 구조

1. Gate 계산하기 : Forget (f_t), Input (i_t), gate(o_t)

Gate는 다음과 같은 함수로 정의되어 0~1 사이의 값을 가지는 벡터

- Gate = $\sigma(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$, σ 는 시그모이드 함수 : 0~1
 - Gate는 최종적으로 0~1사이 값을 가지므로 얼마나 적용할지에 대한 가중치 역할
 - Gate마다 서로 다른 파라미터 (W_{xh} , W_{hh})를 사용하기 때문에 서로 다른 값을 지님
- 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)



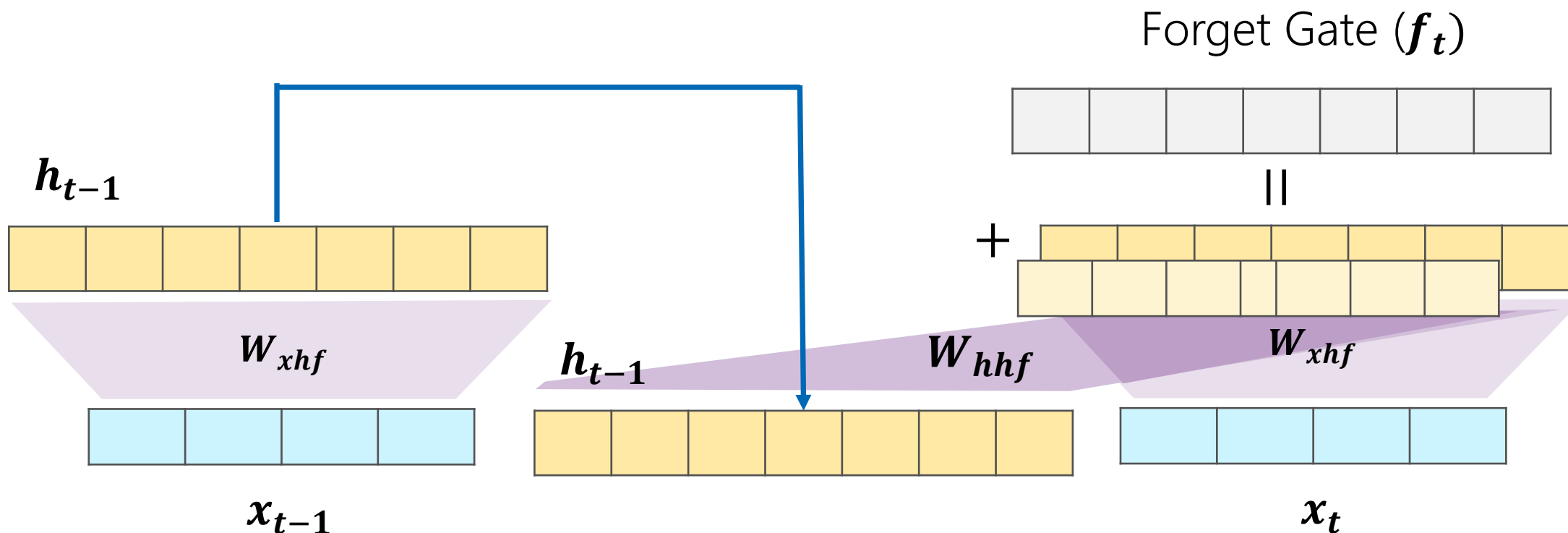
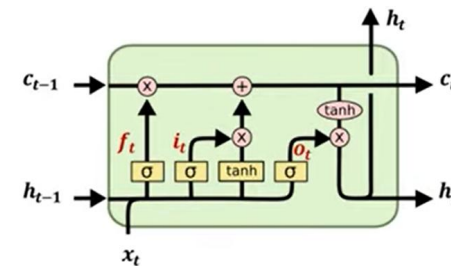
LSTM 구조

1. Gate 계산하기 : Forget (f_t), Input (i_t), gate(o_t)

Forget Gate는 불필요한 과거 정보를 잊기 위한 gate

= 과거 cell state에서 사용하거나 사용하지 않을 데이터에 대한 가중치

- Forget Gate $f_t = \sigma(W_{xhf} x_t + W_{hhf} h_{t-1} + b_{hf})$



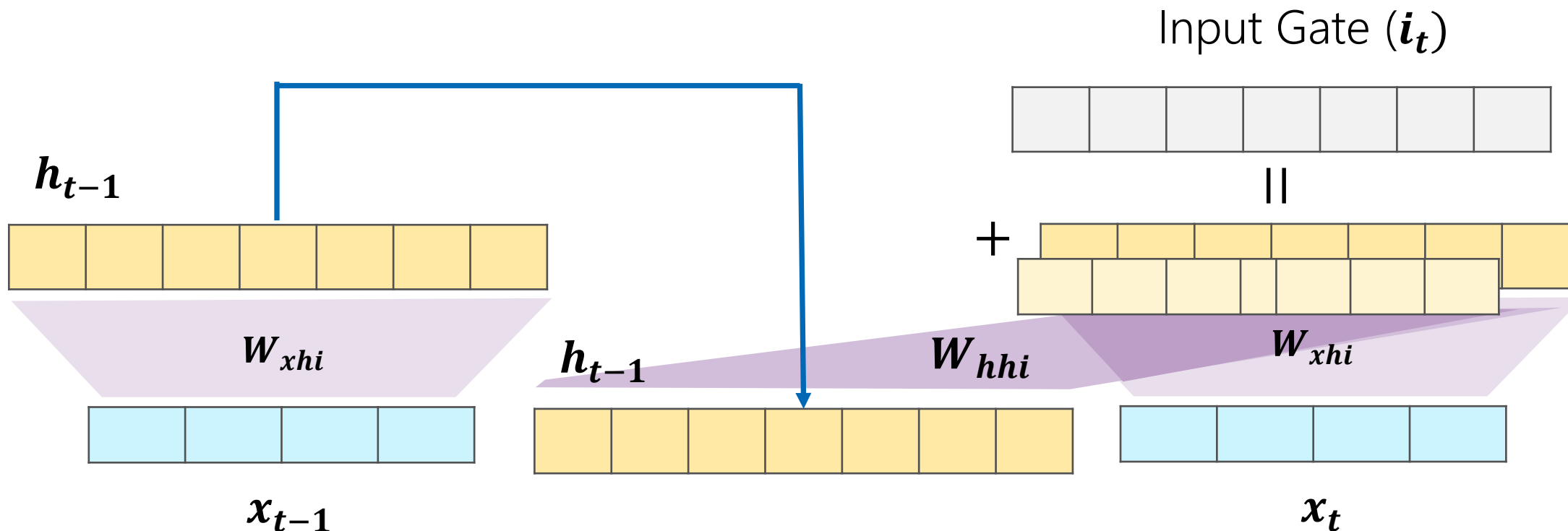
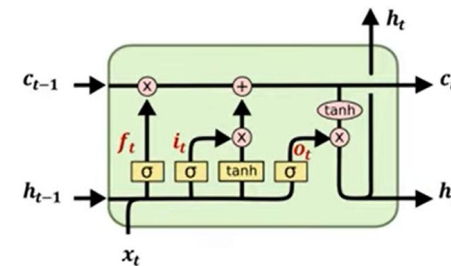
LSTM 구조

1. Gate 계산하기 : Forget (f_t), Input (i_t), gate(o_t)

Input Gate는 현재 정보를 기억하기 위한 gate

= cell state에서 사용할 데이터를 저장하기 위한 가중치

- Input Gate $i_t = \sigma(W_{xhi} x_t + W_{hhi} h_{t-1} + b_{hi})$

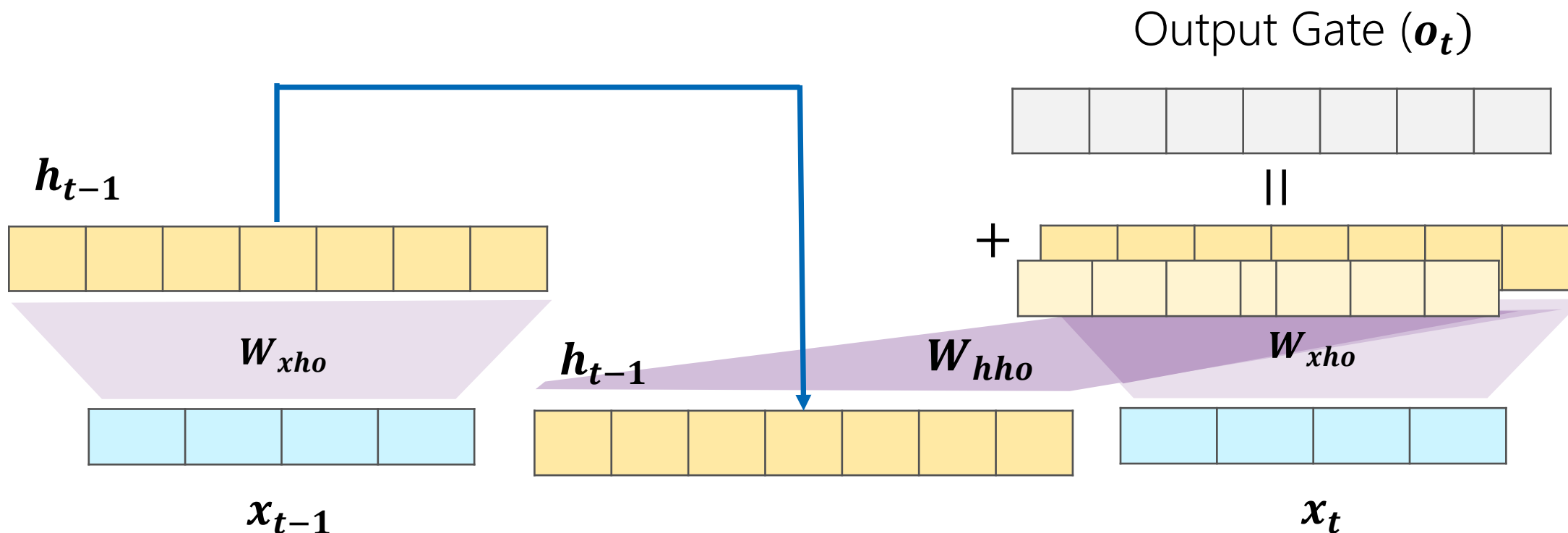
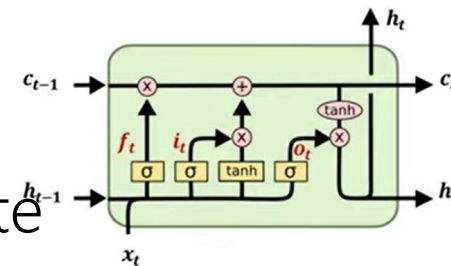


LSTM 구조

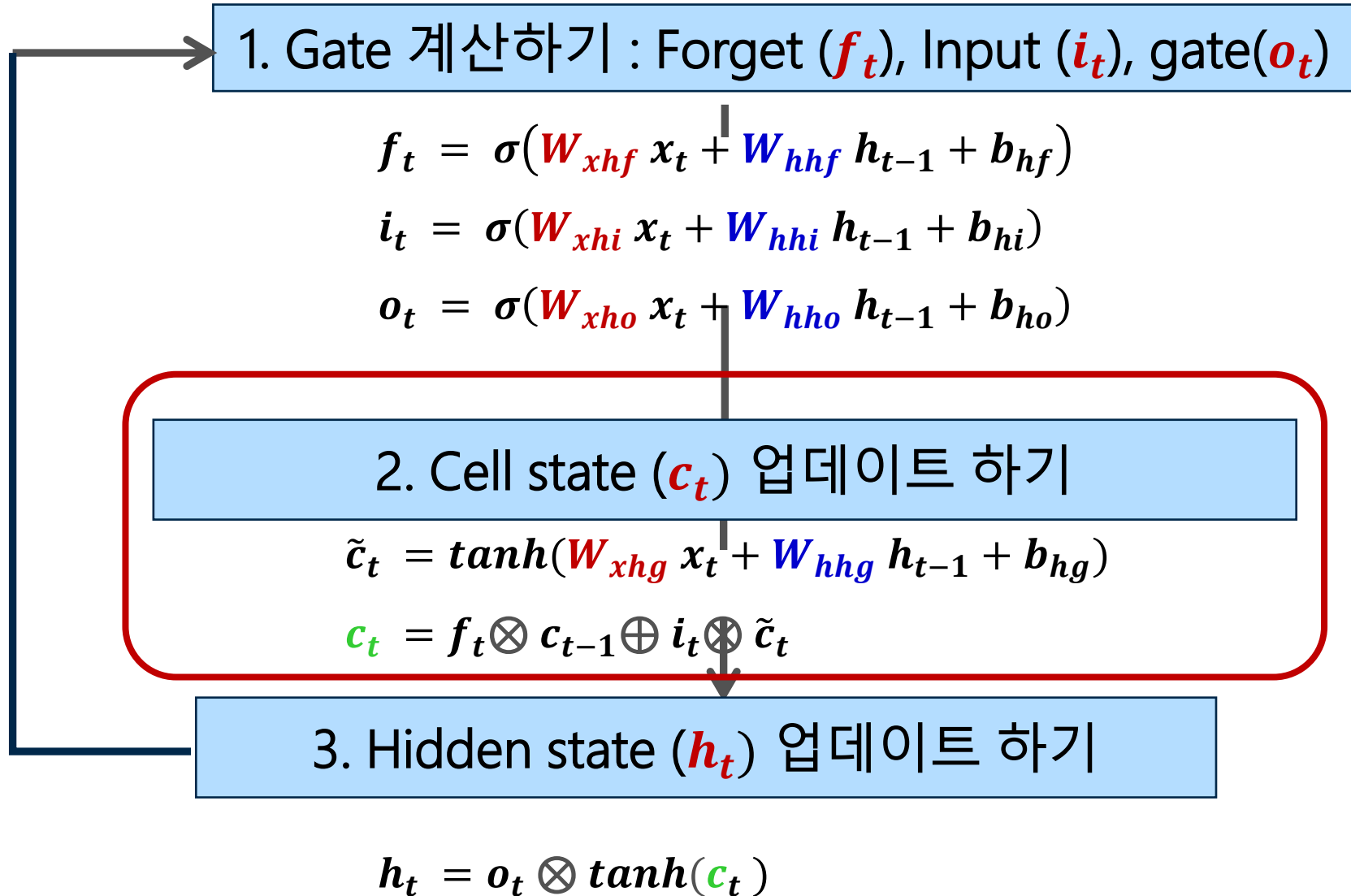
1. Gate 계산하기 : Forget (f_t), Input (i_t), gate(o_t)

Output Gate는 어떤 정보를 output으로 내보낼지 결정하기 위한 gate
= Hidden state에 cell state를 얼마나 반영할 것인지에 대한 가중치

- Output Gate $o_t = \sigma(W_{xho} x_t + W_{hho} h_{t-1} + b_{ho})$



LSTM 구조

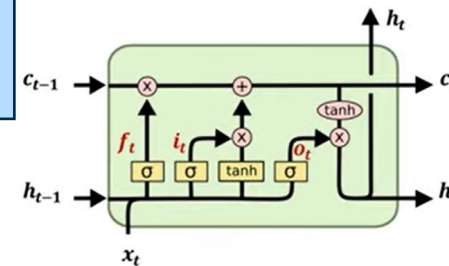


LSTM 구조

2. Cell state (c_t) 업데이트 하기

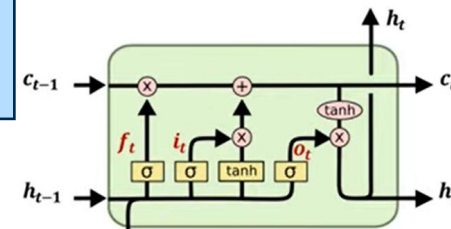
임시 cell state($\tilde{c}_t$)를 생성, 이때 ***tanh*** 를 통해 -1 ~ 1 사이의 값을 가지는 벡터로 산출

- 임시 cell state는 현재 입력값과 과거 hidden state정보에 대한 정보 요약
- 임시 cell state $\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hhg} h_{t-1} + b_{hg})$
- 현 시점에 대한 Cell state (c_t) 를 업데이트 할 때 forget과 input gate를 활용
- Cell state $c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$



LSTM 구조

2. Cell state (c_t) 업데이트 하기



- 현 시점에 대한 Cell state (c_t) 를 업데이트 할 때 forget과 input gate를 활용
- Cell state $c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$
- Forget gate는 불필요한 과거 정보(c_{t-1})를 잊기 위한 gate

다 기억하는 경우

f_t

1	1	1	1	1	1	1
---	---	---	---	---	---	---

c_{t-1}

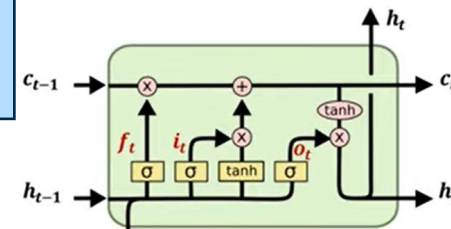
-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1
------	-----	-----	-----	-----	------	------

$f_t \otimes c_{t-1}$

-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1
------	-----	-----	-----	-----	------	------

LSTM 구조

2. Cell state (c_t) 업데이트 하기



- 현 시점에 대한 Cell state (c_t) 를 업데이트 할 때 forget과 input gate를 활용
- Cell state $c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$
- Forget gate는 불필요한 과거 정보(c_{t-1})를 잊기 위한 gate

다 잊는 경우

f_t

0	0	0	0	0	0	0
---	---	---	---	---	---	---

c_{t-1}

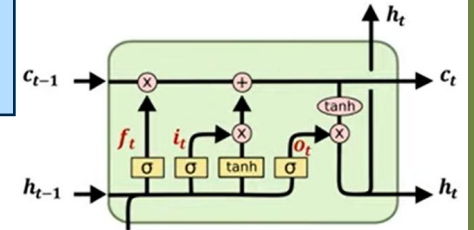
-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1
------	-----	-----	-----	-----	------	------

$f_t \otimes c_{t-1}$

0	0	0	0	0	0	0
---	---	---	---	---	---	---

LSTM 구조

2. Cell state (c_t) 업데이트 하기



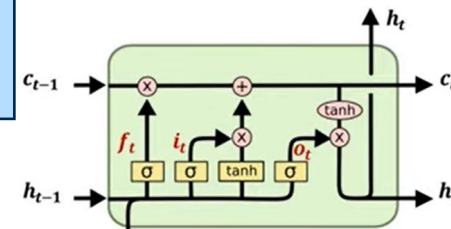
- 현 시점에 대한 Cell state (c_t) 를 업데이트 할 때 forget과 input gate를 활용
- Cell state $c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$
- Forget gate는 불필요한 과거 정보(c_{t-1})를 잊기 위한 gate

일반적인 경우

f_t	0	0.2	0.9	0.1	1	0.2	0.1
c_{t-1}	-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1
$f_t \otimes c_{t-1}$	0	0.02	0.45	0.07	0.9	-0.06	-0.11

LSTM 구조

2. Cell state (c_t) 업데이트 하기



- 현 시점에 대한 Cell state (c_t) 를 업데이트 할 때 forget과 input gate를 활용
- Cell state $c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$
- Input gate는 현재 정보를 기억하기 위한 gate
- $\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hgg} h_{t-1} + b_{hg})$

일반적인 경우

i_t

0.1	0	0.8	0.2	0.8	0.7	1
-----	---	-----	-----	-----	-----	---

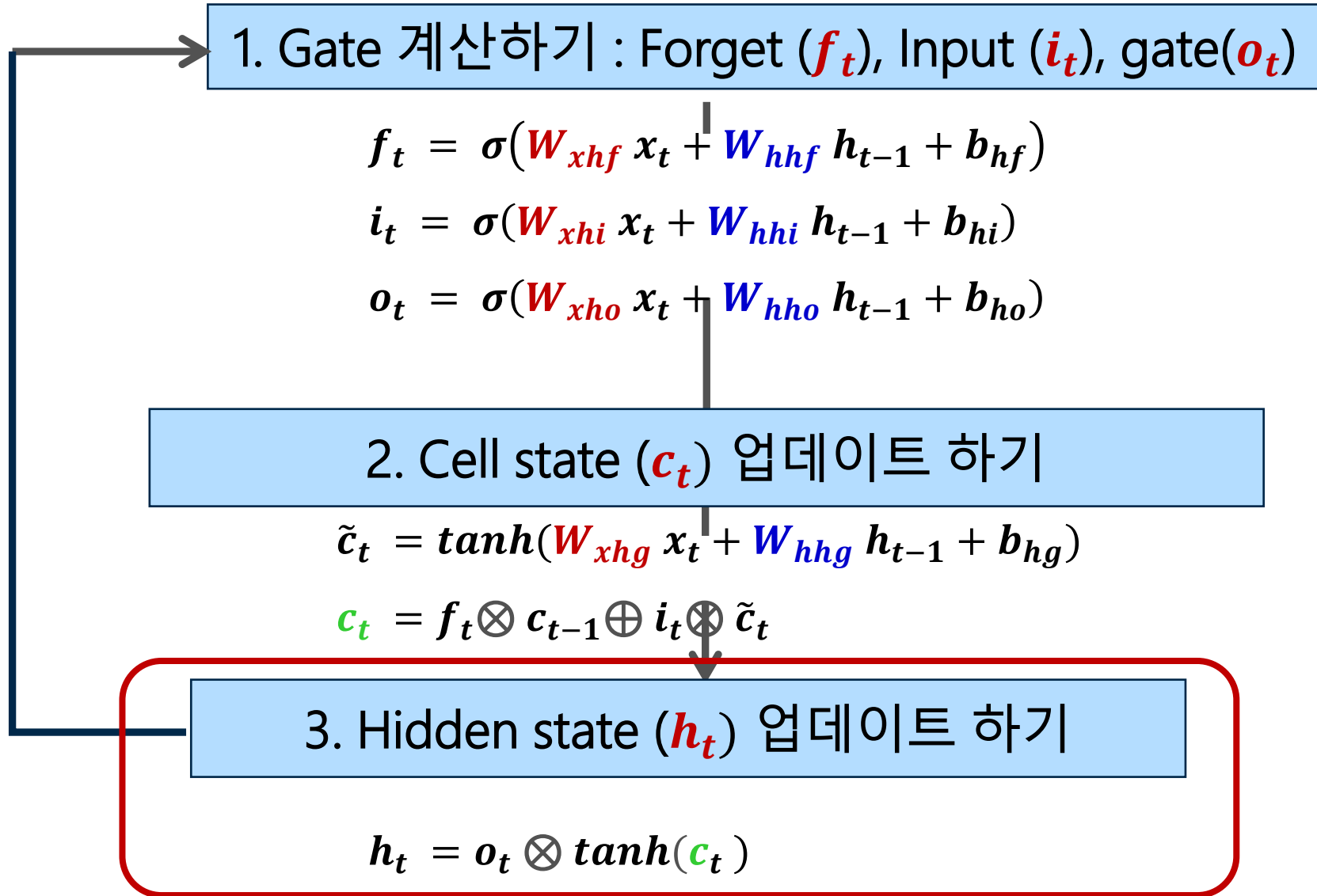
$\tilde{c}_t$

-0.1	0.3	0.6	0.9	0.1	0	2.1
------	-----	-----	-----	-----	---	-----

$i_t \otimes \tilde{c}_t$

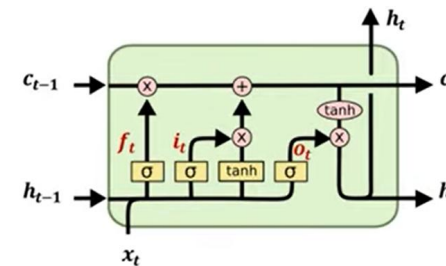
-0.01	0	0.48	0.18	0.08	0	2.1
-------	---	------	------	------	---	-----

LSTM 구조



LSTM 구조

3. Hidden state (h_t) 업데이트 하기



- 최종적으로 단기적 정보 hidden state (h_t)를 업데이트
- $h_t = o_t \otimes \tanh(c_t)$
- h_t 를 이용해서 $\hat{y}_t$ 값을 구할 수 있다. $\hat{y}_t = h_t \times W_{hy}$

o_t

0.5	0.4	0.1	0.9	0.2	0.3	0.8
-----	-----	-----	-----	-----	-----	-----

0~1 사이 값

$\tanh(c_t)$

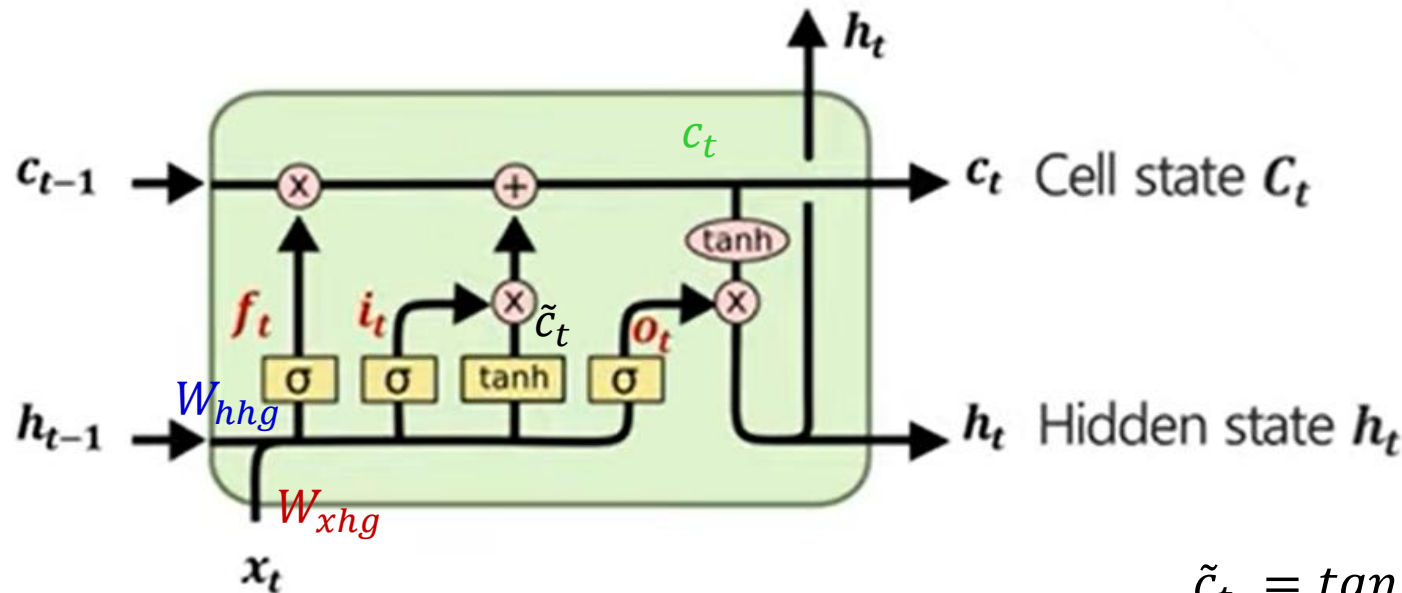
-0.01	0.02	0.73	0.1	0.92	0.01	-0.2
-------	------	------	-----	------	------	------

$o_t \otimes \tanh(c_t)$

-0.005	0.008	0.073	0.09	0.184	0.003	-0.16
--------	-------	-------	------	-------	-------	-------

LSTM - Review

- Long Short Term Memory(LSTM)은 장기 의존성 문제를 완화한 RNN 개선 모델
- Cell state (c_t)구조를 제안하고, 세가지 gate를 추가한 구조
- 세가지 gate : Forget gate(f_t), Input gate(i_t), Output gate(o_t)



$$f_t = \sigma(W_{xhf} x_t + W_{hhf} h_{t-1} + b_{hf})$$

$$i_t = \sigma(W_{xhi} x_t + W_{hhi} h_{t-1} + b_{hi})$$

$$o_t = \sigma(W_{xho} x_t + W_{hho} h_{t-1} + b_{ho})$$

$$\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hhg} h_{t-1} + b_{hg})$$

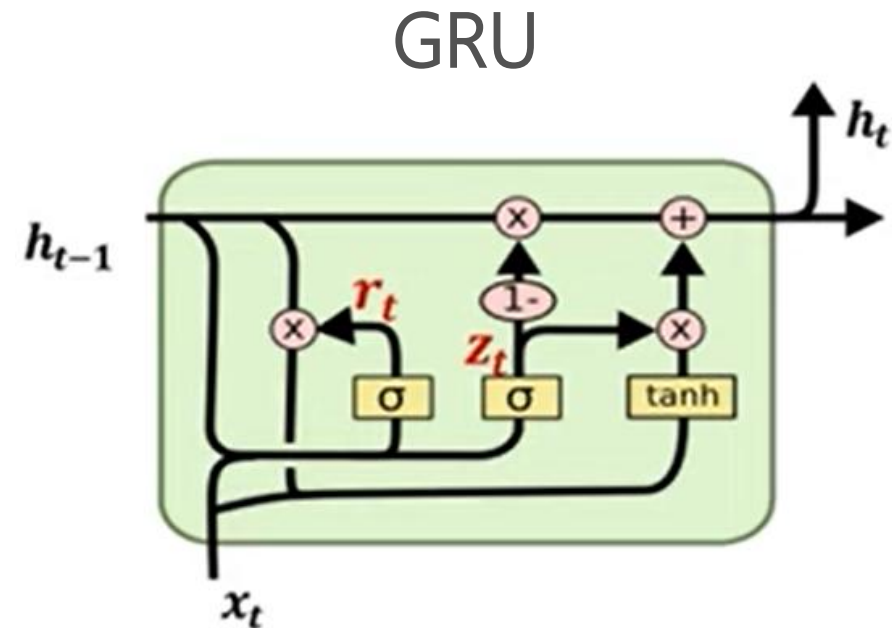
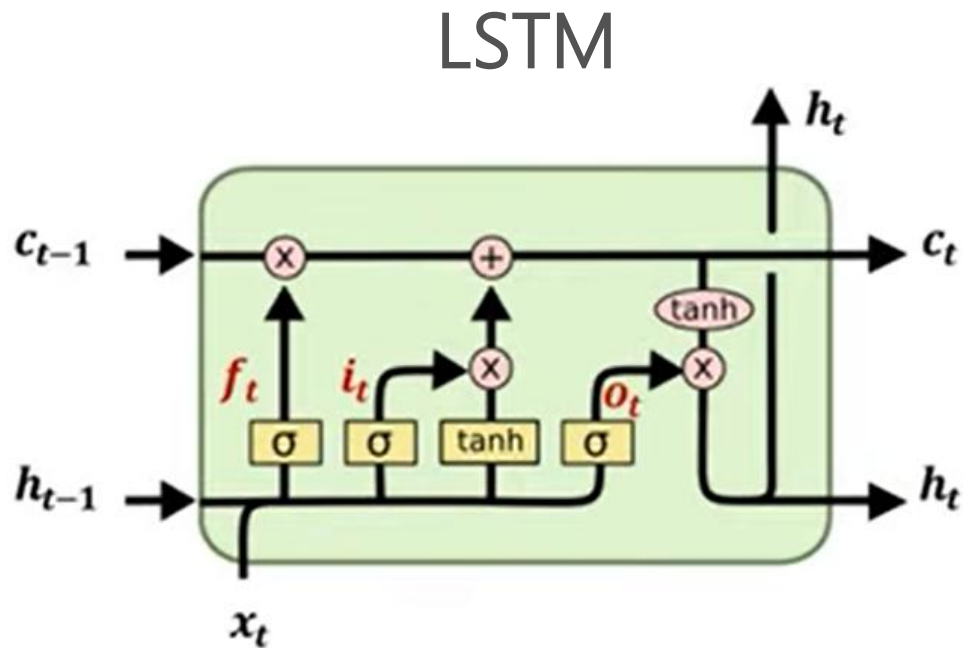
$$c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \tilde{c}_t$$

$$h_t = o_t \otimes \tanh(c_t)$$

$$y_t = g(W_{hy} h_t)$$

LSTM 구조 개선모델 : Gated Recurrent Unit

- 일반적인 LSTM에서 cell state와 세가지 gate가 정의됨 (forget, input, output)
- GRU는 LSTM 구조를 간단하게 개선하여 파라미터 개수를 줄임
- forget gate, input gate를 update gate($\mathbf{z}_t$)로 통합. Output gate를 없애고, reset gate($\mathbf{r}_t$) 정의

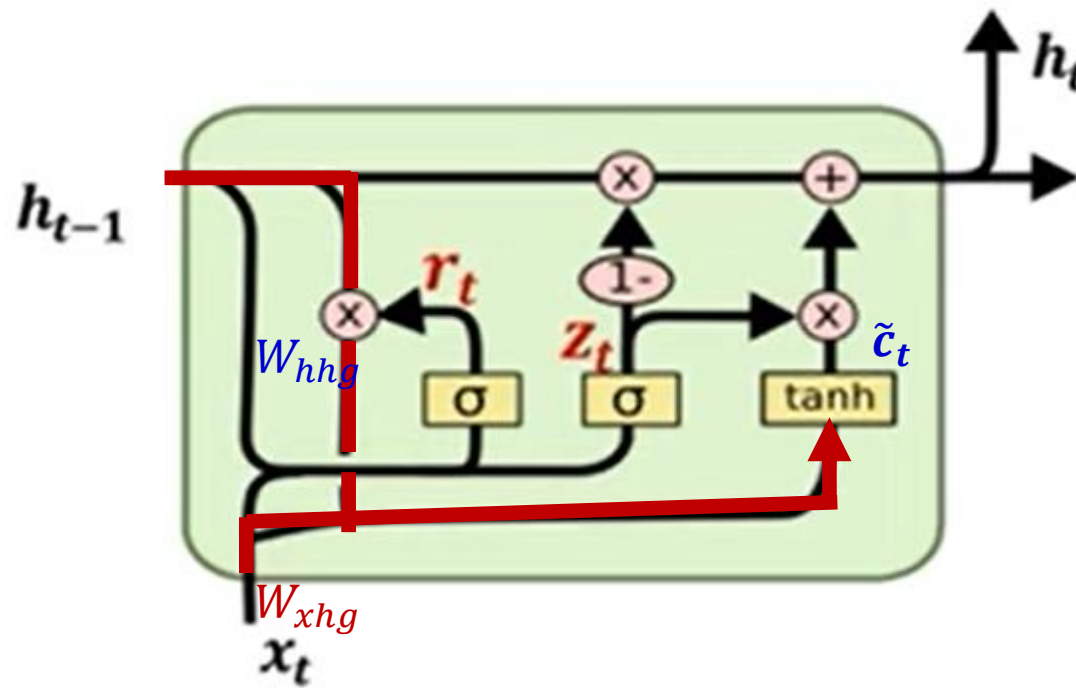


LSTM 구조 개선모델 : Gated Recurrent Unit

Output gate를 없애고, reset gate($\mathbf{r}_t$) 정의. forget gate, input gate를 update gate($\mathbf{z}_t$)로 통합

- reset gate $\mathbf{r}_t = \sigma(W_{xhr} x_t + W_{hhr} h_{t-1} + b_{hr})$
- update gate $\mathbf{z}_t = \sigma(W_{xhz} x_t + W_{hhz} h_{t-1} + b_{hz})$
- 임시 cell state $\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hhg} (\mathbf{r}_t \otimes h_{t-1}) + b_{hg})$

GRU

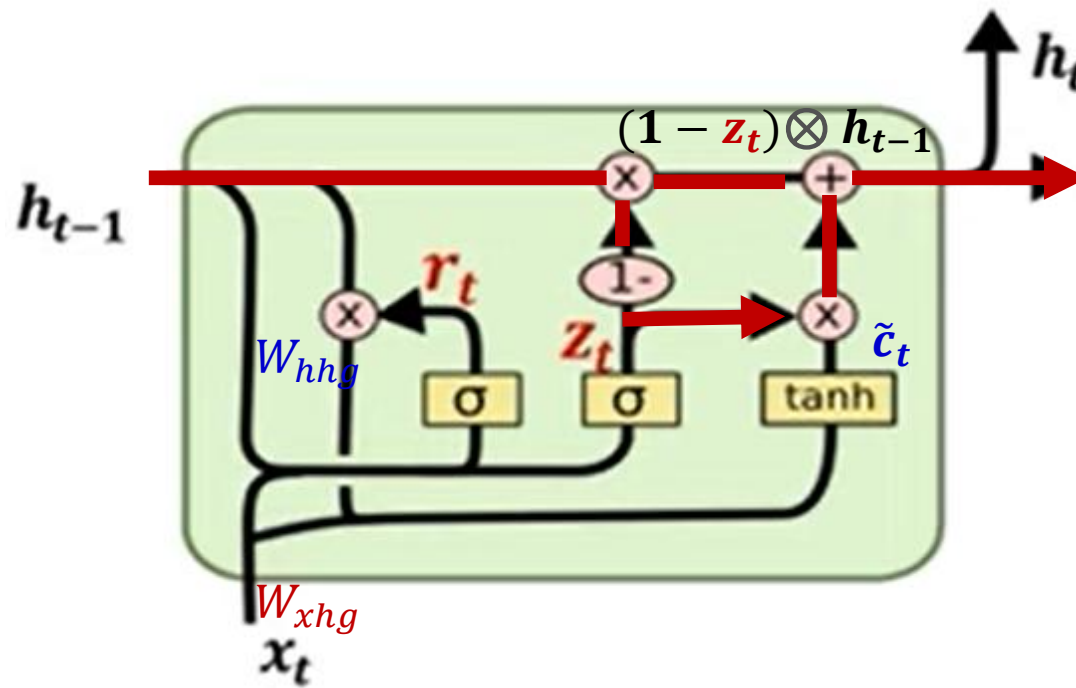


LSTM 구조 개선모델 : Gated Recurrent Unit

Output gate를 없애고, reset gate($\mathbf{r}_t$) 정의. forget gate, input gate를 update gate($\mathbf{z}_t$)로 통합

- reset gate $\mathbf{r}_t = \sigma(W_{xhr} x_t + W_{hhr} h_{t-1} + b_{hr})$
- update gate $\mathbf{z}_t = \sigma(W_{xhz} x_t + W_{hhz} h_{t-1} + b_{hz})$
- 최종 hidden state $\mathbf{h}_t = (1 - \mathbf{z}_t) \otimes \mathbf{h}_{t-1} \oplus \mathbf{z}_t \otimes \tilde{\mathbf{c}}_t$

GRU



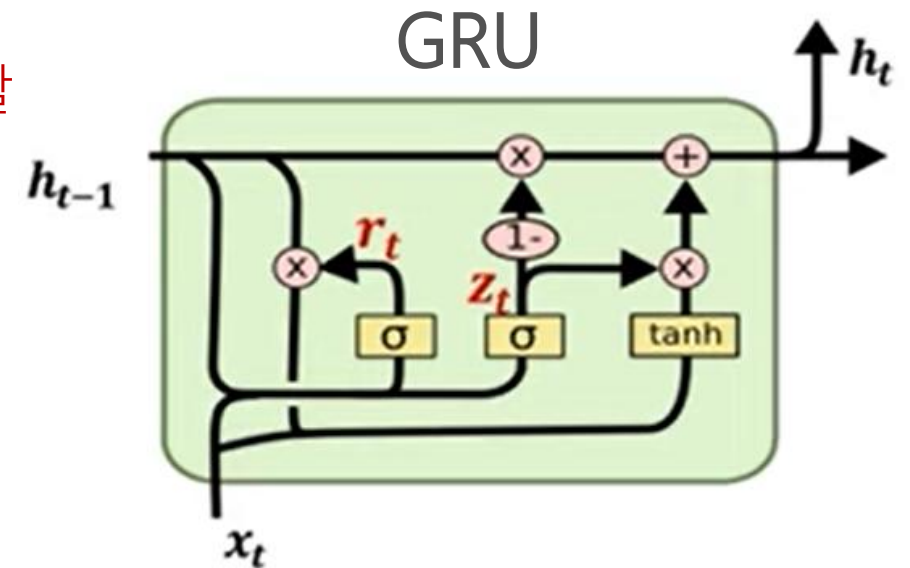
LSTM 구조 개선모델 : Gated Recurrent Unit

Output gate를 없애고, reset gate($\mathbf{r}_t$) 정의. forget gate, input gate를 update gate($\mathbf{z}_t$)로 통합

- reset gate $\mathbf{r}_t = \sigma(W_{xhr} x_t + W_{hhr} h_{t-1} + b_{hr})$
- update gate $\mathbf{z}_t = \sigma(W_{xhz} x_t + W_{hhz} h_{t-1} + b_{hz})$
- 임시 cell state $\tilde{c}_t = \tanh(W_{xhg} x_t + W_{hhg} (\mathbf{r}_t \otimes h_{t-1}) + b_{hg})$

Cell, hidden state를 hidden state로 통합

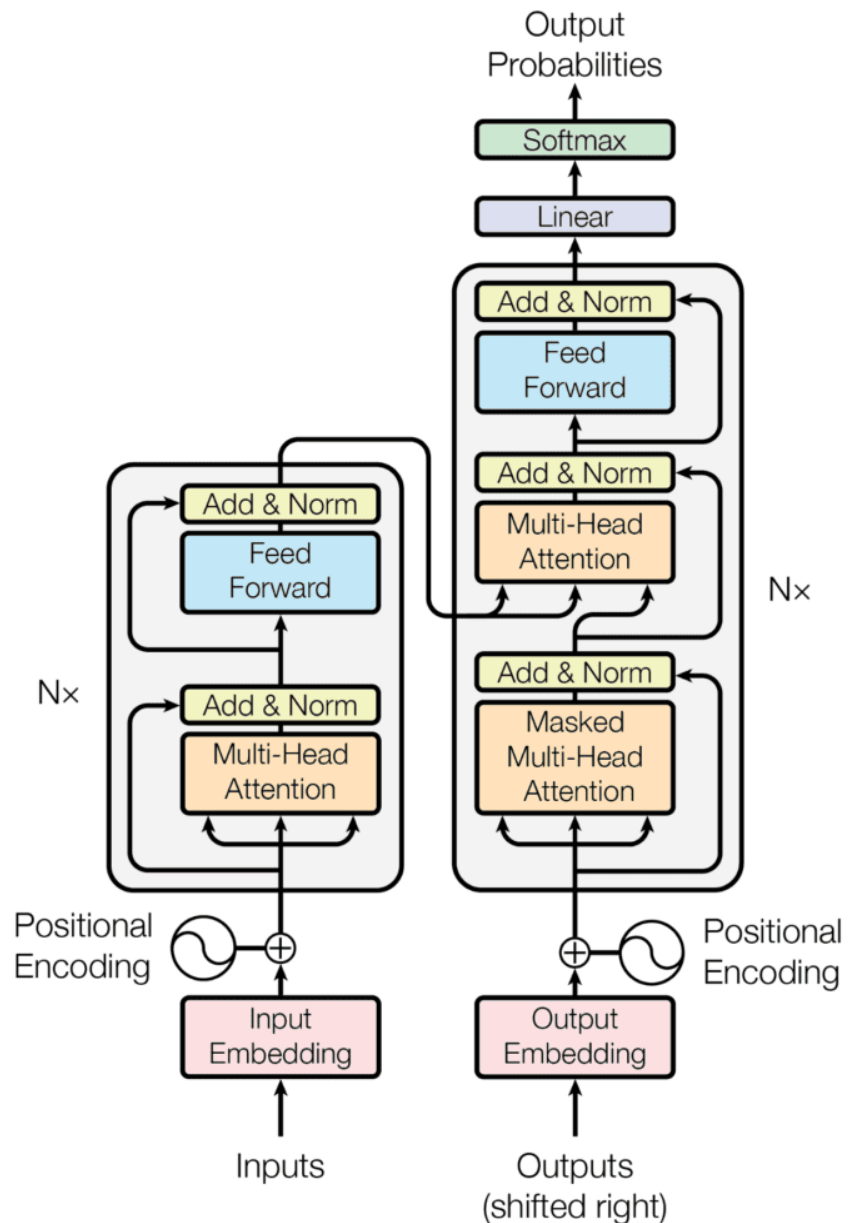
- 최종 hidden state $\mathbf{h}_t = (1 - \mathbf{z}_t) \otimes h_{t-1} \oplus \mathbf{z}_t \otimes \tilde{c}_t$
Forget gate의 역할 Input gate의 역할



Tansformer

AI for Future Workforce

Transformer 구조

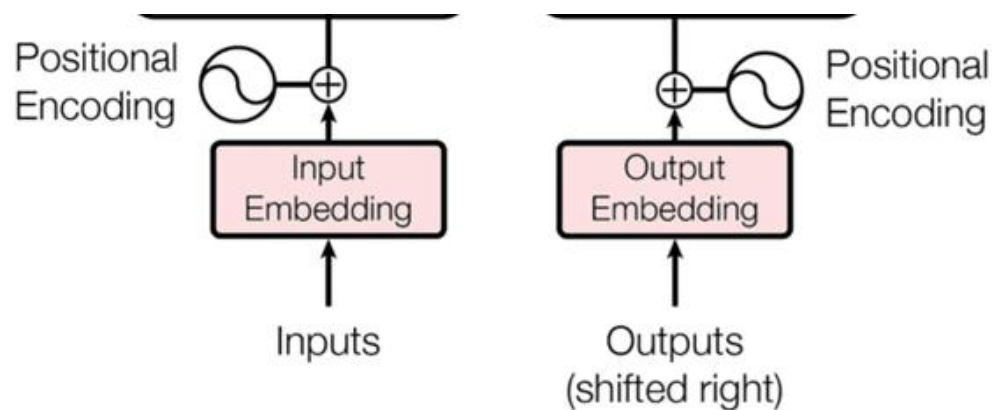


- 2017년에 Google에서 출시
- 병렬 실행이 가능
- Encoder와 Decoder의 구조
 1. Embedding
 2. Positional Encoding
 3. Multi-Head Attention
 4. Feed Forward
 5. Add(Residual Connection)
 6. Norm

참고 [“Attention Is All You Need”](#)

Embedding

- Embedding 자연어를 벡터 형태로 변환
- Transformer에서는 word embedding을 사용
- Output Embedding은 트랜스포머(Transformer) 모델의 디코더(Decoder) 부분에서 사용되며, 주로 자연어 처리(NLP) 작업에서 텍스트 생성(예: 번역, 요약, 텍스트 생성 등)에 사용
- 훈련 중에는 디코더의 입력으로 실제 정답 시퀀스가 사용됩니다. 예를 들어, 번역 모델을 훈련할 때, 소스 언어를 인코더가 처리한 후 디코더는 타겟 언어의 정답 문장을 입력으로 받습니다.
- 추론 중에는 디코더가 이전에 생성한
단어들이 입력으로 사용



Positional Encoding

- 위치 정보를 가지고 있으므로 동시처리, 병렬 연산처리가 가능
- sin, cos 모두 -1~1 사이임으로 정규화 효과.
- Sin 하나만 사용하면 Sin은 주기함수이므로 다른 데이터가 같은 값이 나올 수 있으므로, sin, cos을 번갈아 사용함으로 해서 겹침을 방지
- 단어의미 벡터에 Positional 벡터를 더해서, 의미 + 위치 정보를 가진 입력 벡터를 만든다

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

I

am

a

student

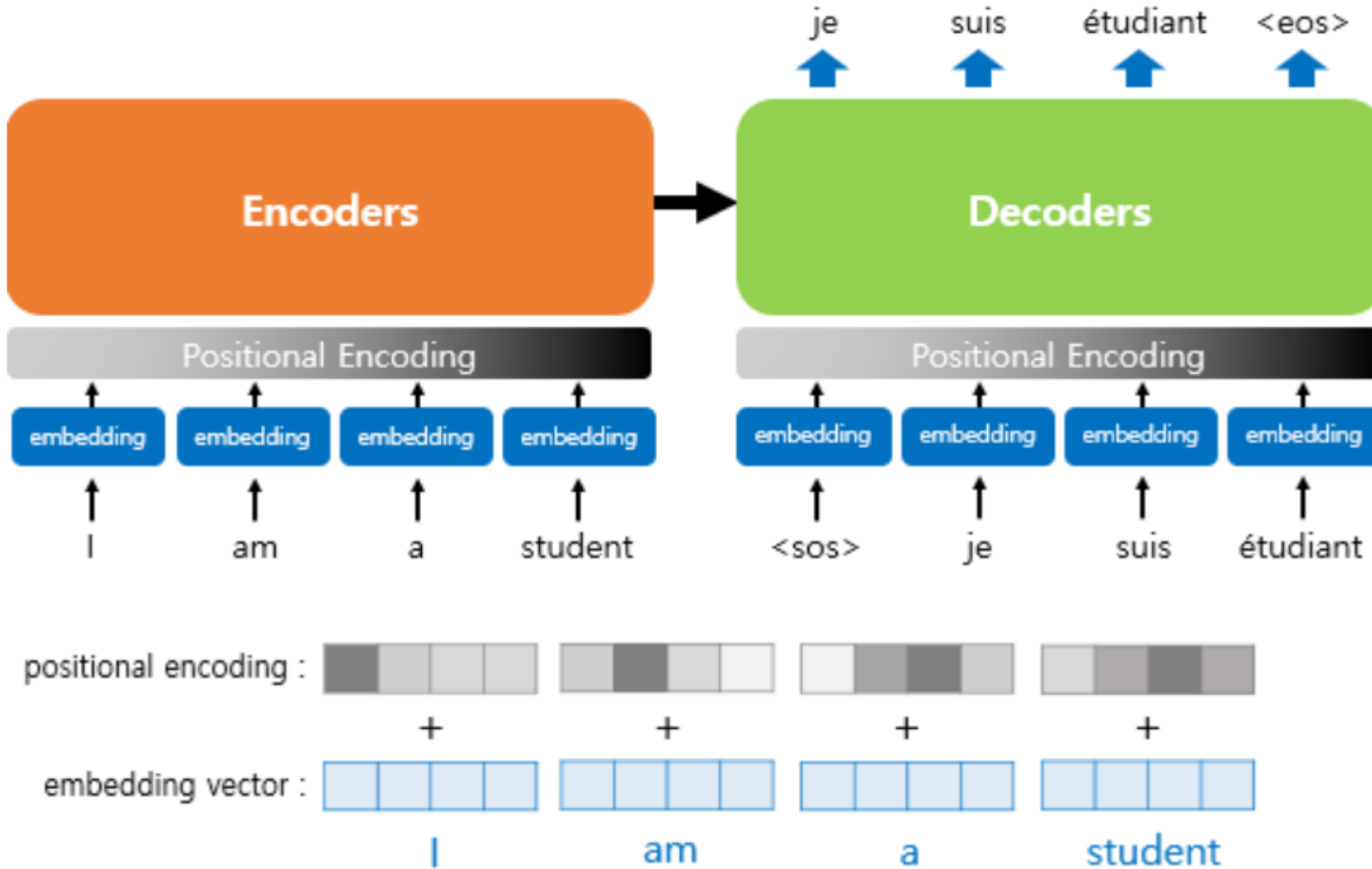
Sin(0)	Cos(0)	Sin(0)	Cos(0)
--------	--------	--------	--------

Sin(1)	Cos(1)	Sin(1)	Cos(1)
--------	--------	--------	--------

Sin(2/..)	Cos(2/..)	Sin(2/..)	Cos(2/..)
-----------	-----------	-----------	-----------

Sin(3/..)	Cos(3/..)	Sin(3/..)	Cos(3/..)
-----------	-----------	-----------	-----------

Positional encoding + embedding vector



출처 : <https://wikidocs.net/31379>

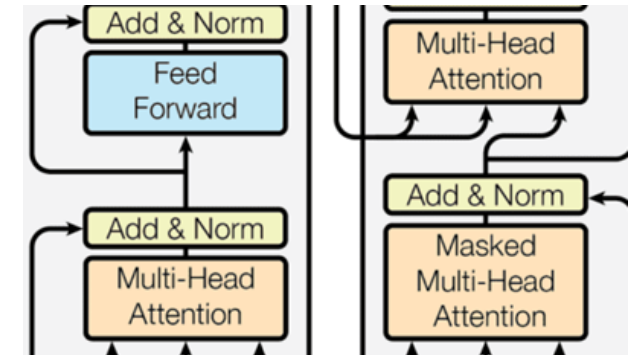
Multi-Head Attention

Attention

- Self-Attention : Self-Attention은 입력 벡터의 가중합을 계산하는 일련의 선형 변환(linear transformation)과 행렬 연산으로 구성.

Weights within an encoder or weights within a decoder

- Query, Key, Value 행렬 구성
- QK^T 는 일반 행렬 곱셈이며 단어들과의 상관관계를 구함
- $\sqrt{d_k}$ 는 정규화 효과
- Self-Attention의 출력은 다음 단계에서 Dense layers와 같은 신경망 층에 입력으로 들어가며, 여기서 비선형성이 도입.



$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad d_k \text{ 는 Key 벡터의 차원 수}$$

Multi-Head Attention

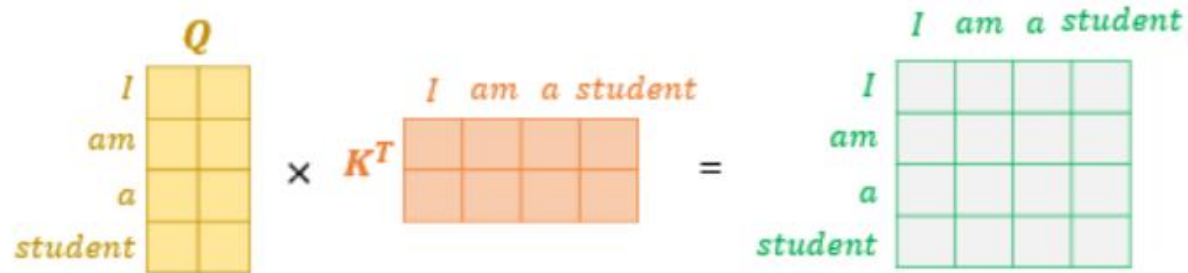
$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{Q}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{K}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{K} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{V}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

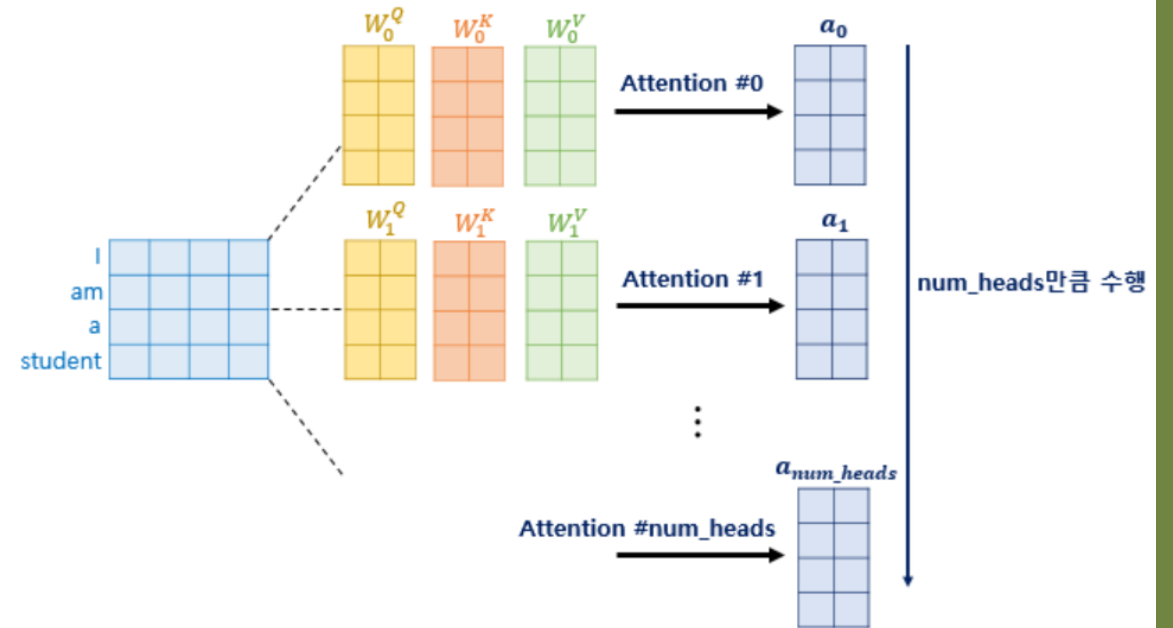
출처 : <https://jalammar.github.io/illustrated-transformer/>

Multi-Head Attention



$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) \times V = \text{Attention Value Matrix } a$$

Diagram illustrating the scaled dot-product attention mechanism. A query matrix Q (4x2, yellow) is multiplied by the transpose of a key matrix K^T (4x4, orange). The result is passed through a softmax function, scaled by $\sqrt{d_k}$, and then multiplied by a value matrix V (4x4, green) to produce the final Attention Value Matrix a (4x4, blue).



Multi-Head Attention

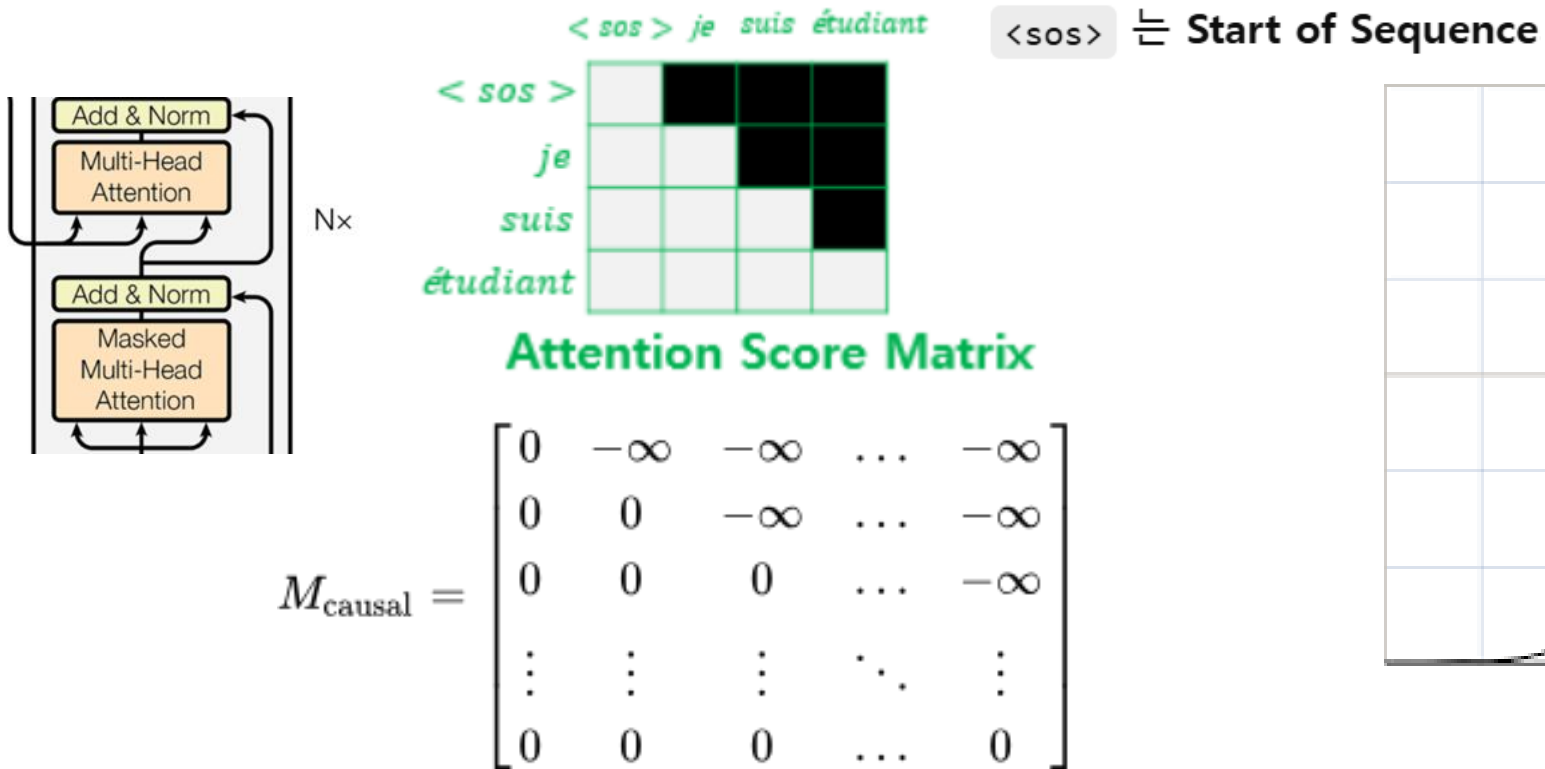
Attention

- 예를 들어, "The cat sat on the mat"라는 문장에서 Self-Attention은 "cat"이 "sat"와 강하게 관련되어 있음을 학습.
- 이렇게 단어들 간의 관계를 학습함으로써 모델은 문맥을 이해할 수 있으며, 이는 특히 문장 구조가 복잡하거나 장기적인 의존성을 다룰 때 매우 유리
- Self-Attention 메커니즘은 순차적인 처리를 요구하지 않기 때문에 모든 단어에 대해 병렬로 상관관계를 계산
- Self-Attention 메커니즘을 통해 단어들 간의 상관관계를 학습한 후, 그 결과를 Dense Layer로 전달하는 방식은 트랜스포머 모델의 성능을 크게 향상
- Multi-Head 이므로 N개의 K, Q, V 임계치 배열이 만들어지고, 서로 다른 상관관계를 학습 (CNN에서 여러 개의 Kernel을 도입하는 개념과 비슷)
- Dense Layer는 비선형성을 도입하여 Self-Attention에서 학습된 상관관계 정보를 더욱 풍부하고 복잡한 특징으로 변환

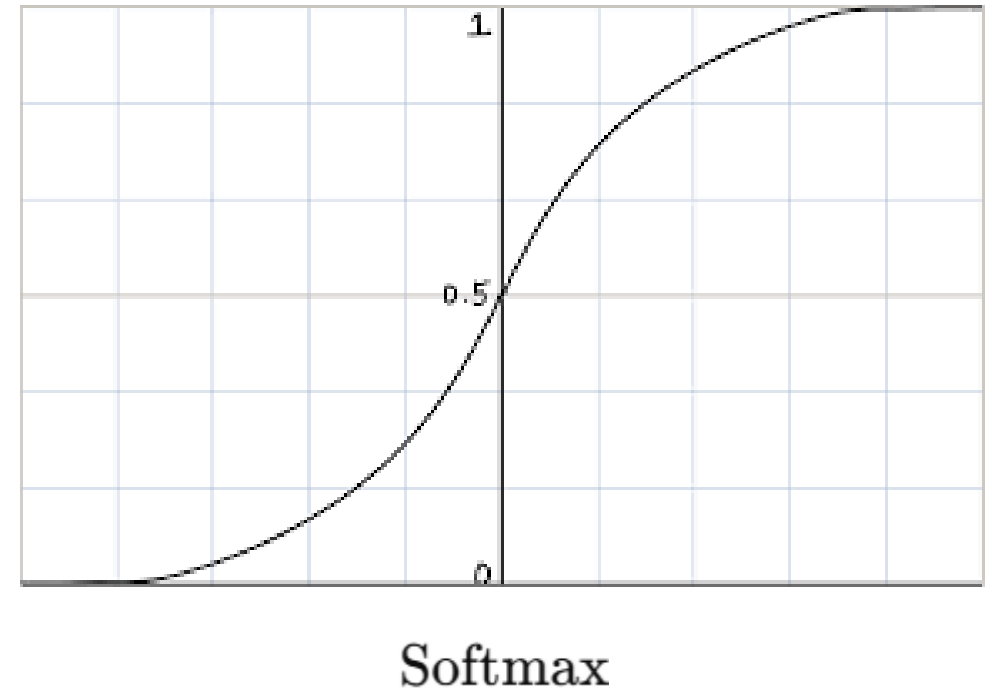
Masked Multi-Head Attention

Attention

- 디코더의 Masked 의미는 모델이 미래의 단어를 보지 못하게 하는 의미



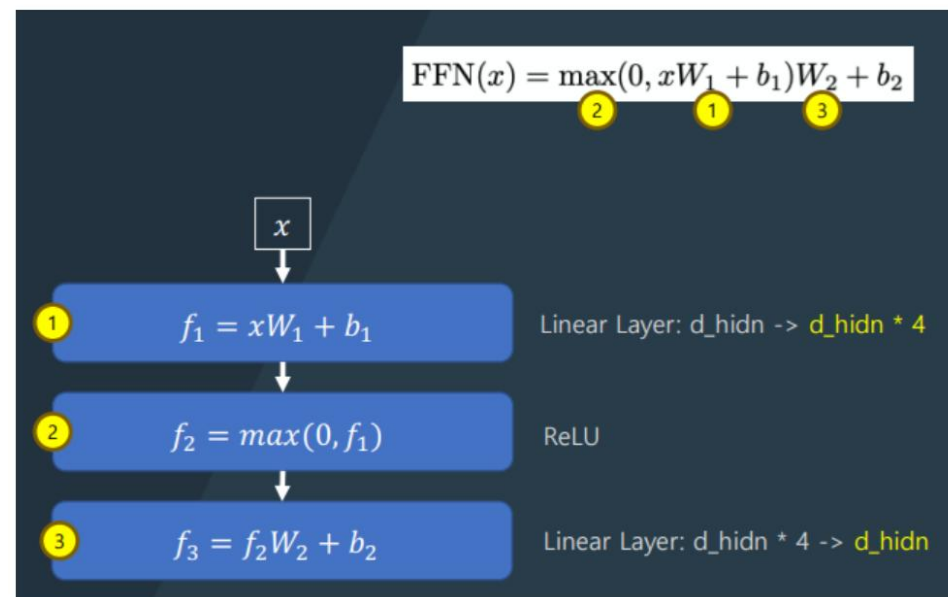
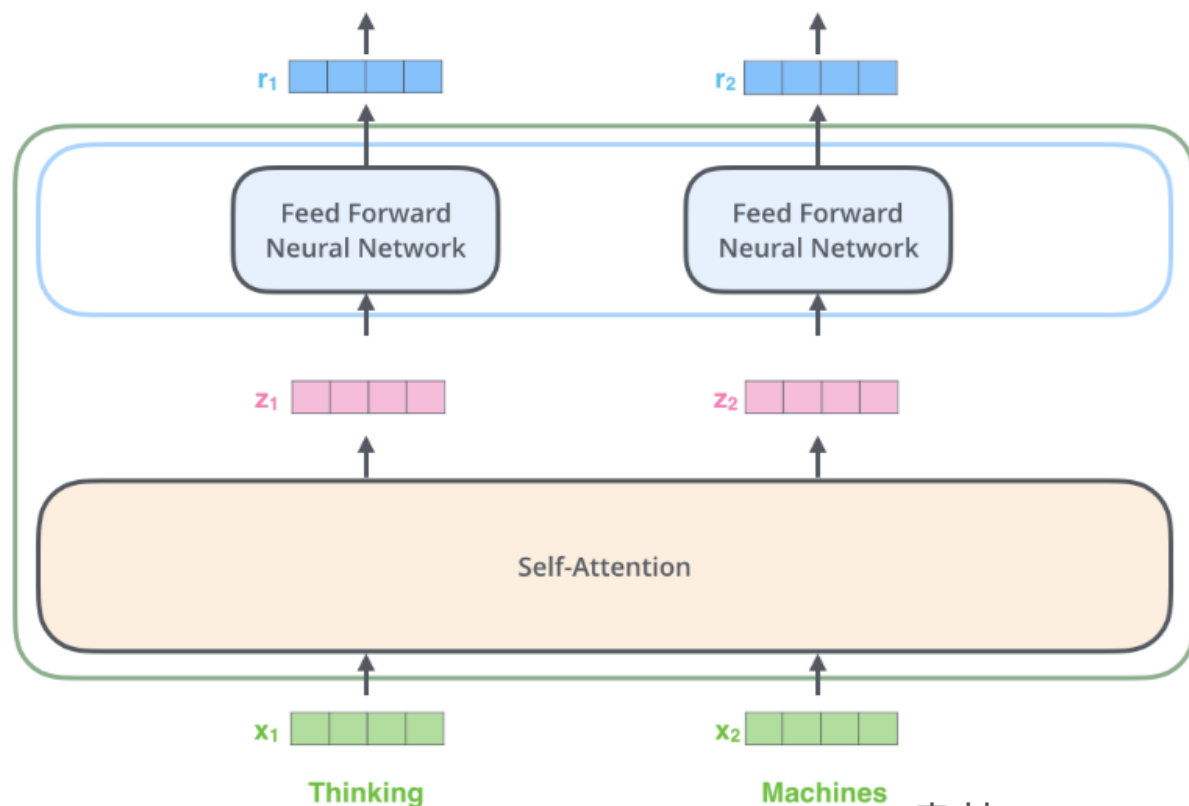
$$S' = S + M_{\text{causal}}$$



출처 : <https://wikidocs.net/31379>

Feed Forward

- 트랜스포머 모델의 Feed Forward 층은 Self-Attention 층의 출력을 비선형적으로 변환하여 모델의 표현력을 강화
- 이 층을 통해 모델은 단순한 상관관계 이상의 복잡한 패턴을 학습할 수 있으며, 각 위치에서 독립적으로 계산이 이루어져 병렬 처리의 장점도 살릴 수 있다.



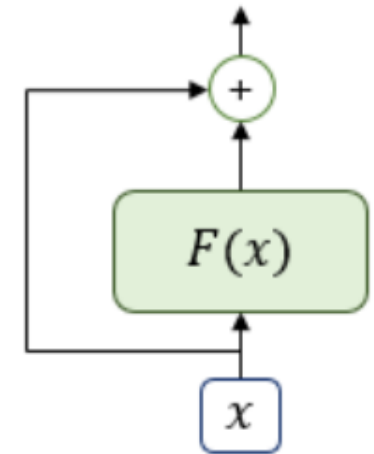
출처 : <https://wikidocs.net/31379>

출처 : <https://jalammar.github.io/illustrated-transformer/>

Add

- Add 단계에서의 Residual Connection 개념을 사용
- 트랜스포머 모델의 각 인코더 및 디코더 블록에서, Self-Attention 층이나 Feed Forward 층을 통과한 출력에 Residual Connection을 사용하여 원래의 입력을 더합니다.
- 수식으로 표현하면: $\text{Output} = \text{Layer Output} + \text{Layer Input}$
 - Layer Output은 Self-Attention이나 Feed Forward 층을 통과한 출력
 - Layer Input은 그 층에 주어진 원래의 입력입니다.
- 딥러닝 모델이 깊어질수록, 즉 층이 많아질수록 역전파 과정에서 기울기(gradient)가 소실되는 문제가 발생할 수 있습니다.
- Residual Connection은 이 문제를 완화하여 더 깊은 네트워크를 안정적으로 학습할 수 있게 합니다. 원래의 입력 값을 출력에 더해줌으로써, 역전파 시 기울기가 안정적으로 전파될 수 있습니다.

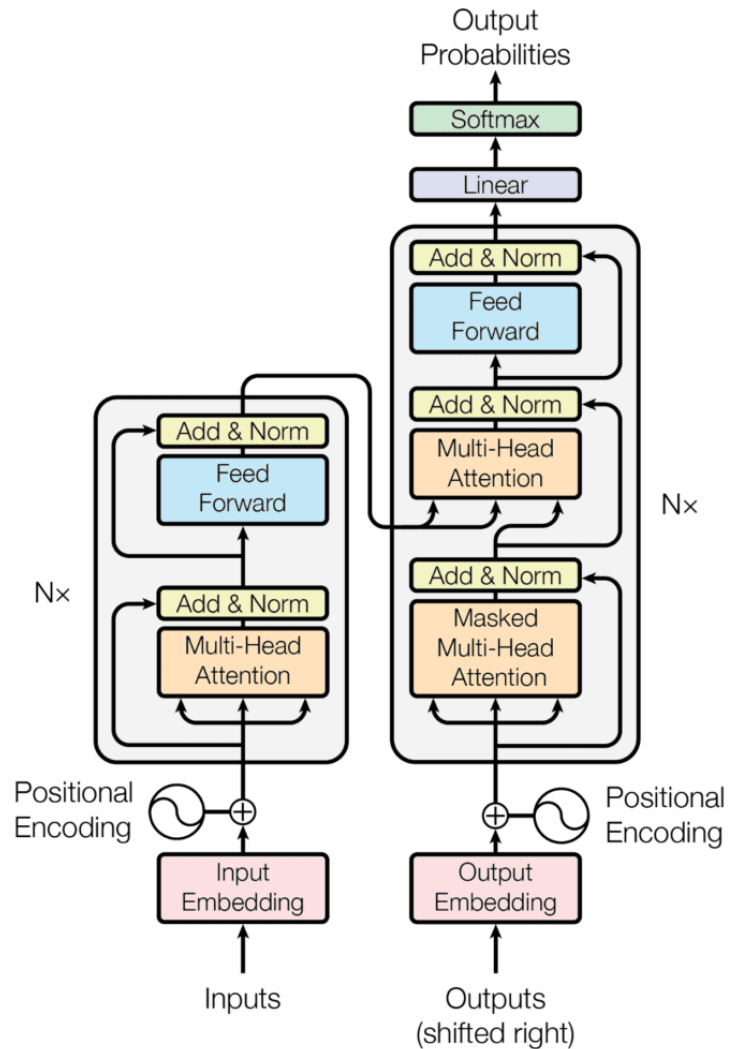
$$H(x) = x + F(x)$$



Norm

- Norm 단계 (Layer Normalization) : Residual Connection을 통해 더한 출력은 Layer Normalization을 통해 정규화됩니다.
- Layer Normalization은 네트워크의 출력을 정규화하여 학습을 안정화하고, 과적합(overfitting)을 방지하는 데 도움을 줍니다.

Transformer 구조



- 2017년에 Google에서 출시
- 병렬 실행이 가능
- Encoder와 Decoder의 구조

참고 [“Attention Is All You Need”](#)



intel. digital readiness

